

MICROCONTROLLERS

Labo 0

Naslag

Gegenereerd op 31/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Basisbegrippen

[Wat is een microcontroller?](#)

[Hard- en software](#)

[Setup en loop](#)

[Een programma uploaden](#)

Programmeerconcepten

[De spelregels van C++](#)

[Constanten, variabelen en gegevenstypes](#)

[Wiskundige operatoren](#)

[Selecties](#)

[Iteraties](#)

[Functies en parameters](#)

I/O & signalen

[pinMode\(\)](#)

[digitalRead / digitalWrite](#)

[Pull up en pull down weerstanden](#)

[Sourcen en sinken](#)

[De wet van Ohm](#)

[Weerstand en impedantie](#)

Debuggen & robuustheid

[Debouncen](#)

[Debuggen](#)

Wat is een microcontroller?

Een microcontroller is een computer die in één chip zit en die jij zelf programmeert. In het eerste jaar van het modeltraject schreef je C#-programma's voor een pc in Technisch programmeren, en programmeerde je een Siemens LOGO in Logische sturingen. Dit semester komen daar PLC's en microcontrollers bij. Deze pagina zet die drie soorten toestellen naast elkaar, zodat je weet waarvoor een microcontroller dient en waarvoor niet.

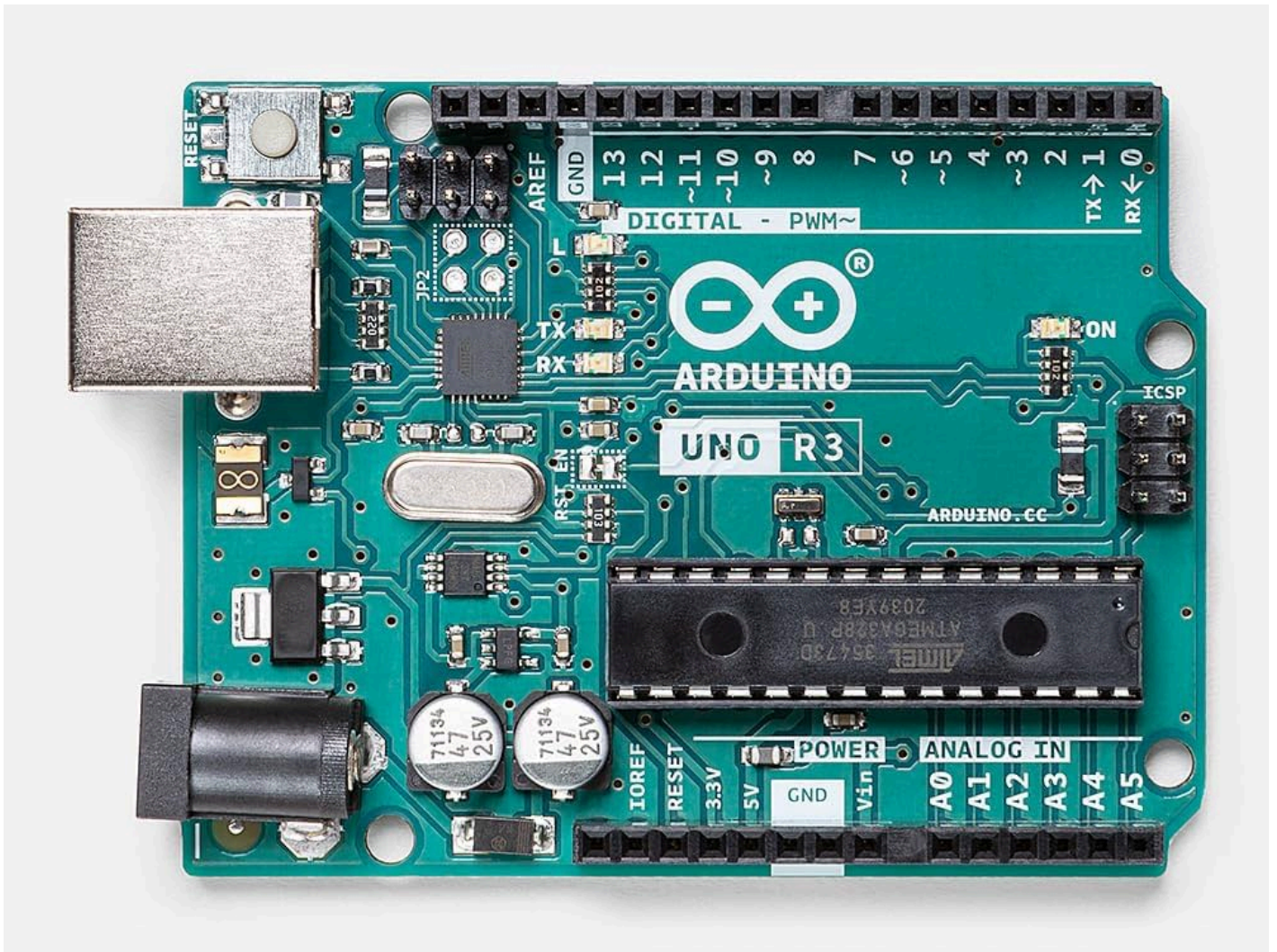
Een computer op één chip

Een **microcontroller** is een volledige computer in één chip. De processor, het geheugen en de aansluitingen naar de buitenwereld zitten samen in één behuizing. Een pc heeft daarvoor een moederbord met losse onderdelen, een harde schijf en een grafische kaart nodig.

Zo'n chip heeft nog wel wat rondom zich nodig voor je hem kan gebruiken: een voeding, een manier om je programma binnen te krijgen en aansluitingen waar je draden in kan steken. Een Arduino-bord levert dat alles.

DE ARDUINO IS NIET DE MICROCONTROLLER

Op een Arduino Uno is de microcontroller de langwerpige zwarte chip met een rij aansluitingen langs beide kanten, een **ATmega328P**. De rest van de printplaat dient om die chip bruikbaar te maken.



☑ Een Arduino Uno. De brede zwarte chip onderaan, met ATMEGA328P erop, is de microcontroller zelf.

De drie toestellen naast elkaar

Op alle drie draait een programma dat jij geschreven hebt, maar verder werken ze verschillend. De tabel zet de belangrijkste verschillen op een rij; de twee stukken erna leggen uit wat die voor jou betekenen.

	Laptop	Arduino Uno	Siemens LOGO of PLC
Wat er draait	Windows, en daarbovenop tientallen programma's tegelijk	Geen besturingssysteem, alleen jouw programma	Alleen jouw programma, op vaste firmware van de fabrikant
Start en stop	Jij start het programma, en het eindigt	Start bij het aanzetten en stopt nooit	Draait cyclisch zolang het toestel in RUN staat
Waar in je programmeert	C# in Visual Studio	C++ in de Arduino IDE	Functieblokken of ladderlogica
Fouten zoeken	Breakpoints en een console	Alleen tekst via de seriële monitor	Live meekijken in de programmeersoftware
Schermb en bediening	Schermb, toetsenbord en muis	Niets	Een klein scherm met knoppen, of een apart bedieningspaneel
Geheugen	Enkele GB werkgeheugen, honderden GB opslag	2 KB werkgeheugen, 32 KB voor je programma	Je telt blokken in plaats van bytes
Snelheid	Enkele GHz, meerdere kernen	16 MHz, één kern	Een cyclustijd van enkele milliseconden, bewaakt door het toestel
Contact met de buitenwereld	Via het besturingssysteem en drivers	Rechtstreeks via de pinnen, met echte stroom en spanning	Via klemmen: beveiligde ingangen en uitgangen die vermogen schakelen
Waar het thuisheert	Op een bureau	Ingebouwd in het product zelf	In een schakelkast, op een DIN-rail

Laptop, Arduino Uno en PLC naast elkaar

Wat dat betekent tegenover je laptop

De oneindige lus. Op je laptop reageert een **EventHandler** in C# op het klikken van een knop: je code doet iets, geeft de controle terug, en het programma wacht tot er weer iets gebeurt. Een programma op een microcontroller blijft draaien zolang er spanning op de chip staat, en eindigt dus nooit.

Debuggen. Werkt je C#-programma niet, dan zet je een breakpoint en kijk je stap voor stap mee. Op een microcontroller kan dat niet, en is de seriële verbinding meestal je enige manier om te **debuggen**.

De pinnen. Tussen jouw code en de hardware zit geen besturingssysteem: `digitalWrite()` zet letterlijk 5 V op een aansluiting van de chip. Daarom hoort er elektronica bij dit vak, en ben je zelf verantwoordelijk voor wat je aan elke pin hangt.

Het geheugen. De ATmega328P heeft 2 kB werkgeheugen en 32 kB programmeergeheugen. Dat is kilobyte en niet megabyte, en daarom kies je bij elke variabele bewust een gegevenstype. Zie [Constanten, variabelen en gegevenstypes](#).

Wat dat betekent tegenover een LOGO of PLC

Een PLC is een industriële stuurkast die machines aanstuurt, en dit semester ga je er in het vak PLC zelf mee aan de slag. De Siemens LOGO uit Logische sturingen is een kleinere uitvoering van hetzelfde idee.

De cyclus. Een PLC leest zijn ingangen, werkt het programma af, schrijft zijn uitgangen en begint opnieuw. Dat is dezelfde vorm als de `loop()` in een Arduino-programma. Het verschil is dat een PLC dat lezen en schrijven zelf doet en ook zijn cyclustijd bewaakt; op een Arduino schrijf je het zelf met `digitalRead()` en `digitalWrite()`, en wordt de cyclustijd niet bewaakt.

De klemmen. Op een LOGO schroef je een knop op een ingang en een lamp op een uitgang, en dat werkt meteen: die aansluitingen zijn beveiligd en de relaisuitgangen mogen 230 V schakelen. Een pin van een microcontroller heeft die beveiliging niet en kan maar een kleine stroom leveren, dus wat de LOGO voor jou regelde, bouw je hier zelf op je breadboard.

WAARVOOR JE EEN MICROCONTROLLER KIEST

Een microcontroller kost een paar euro en is klein genoeg om in het product zelf te zitten: in je afstandsbediening, je wasmachine en je fietscomputer zit zo'n chip. Je programmeert hem in C++ in plaats van in functieblokken, en je bepaalt zelf wat er aan elke pin hangt. Voor industriële toepassingen waar vermogen geschakeld wordt, gebruik je een PLC.

Hard-en software

Een overzicht van wat je nodig hebt voor dit vak: welke hardware je aanschaft, en welke software je installeert om je Arduino te programmeren.

Hardware

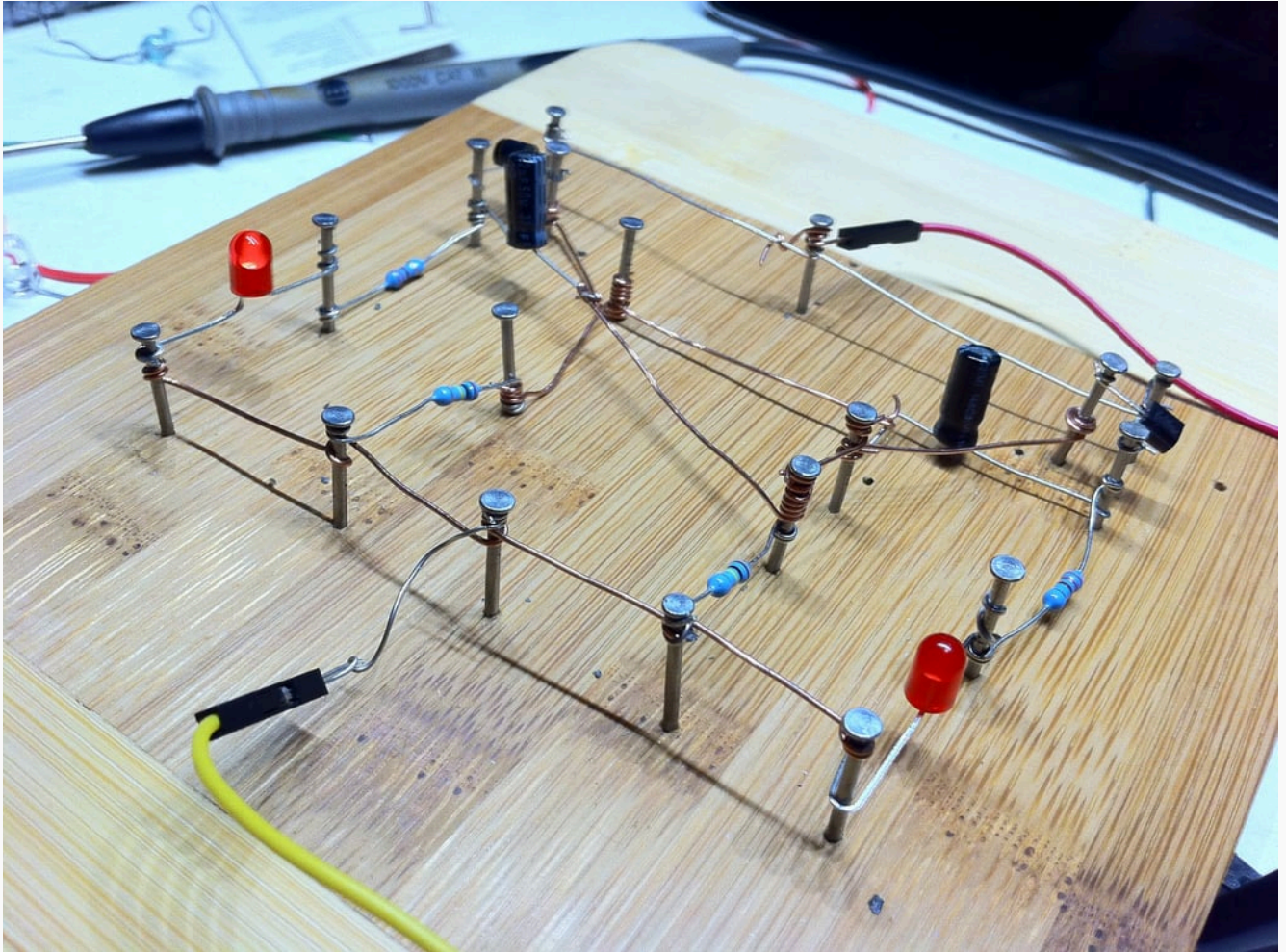
- **Arduino starterkit** (Velleman kit) via [Depraets](#)
- Tijdens de verplichte lessen worden ook **extra componenten** zoals schuifregisters, transistoren, motoren, ... uitgedeeld

Software

- Download de **Arduino Integrated Development Environment (IDE)** via [Academic Software](#) of <https://www.arduino.cc>
 - Bij de installatie zet je alle opties aan en installeer je ook de extra drivers
- Registreer je op [TinkerCAD](#) met je HOGENT-e-mailadres
 - TinkerCAD is een **simulatieomgeving** die we gebruiken voor oefeningen met componenten die je zelf niet ter beschikking hebt

i WAAR DE NAAM BREADBOARD VANDAAN KOMT

In je kit zit een **breadboard**, de witte plaat met rijen gaten waarop je je schakelingen bouwt. De eerste radiobouwers hadden zoiets nog niet: die sloegen spijkers in een houten broodplank en wikkelden hun draden daaromheen. Rond 1970 kwam de plastic versie op de markt waar je de draden gewoon in steekt, en de naam ging mee.



☑ Een schakeling gebouwd zoals het vroeger ging: spijkers in een houten plank, met de draden en de componenten ertussen.
Bron: Adafruit Learning System

Setup en loop

Een Arduino-programma bestaat altijd uit twee functies: **setup** en **loop**.

CPP

```
1 void setup()  
2 {  
3     // Hier komt code die maar één keer wordt uitgevoerd, bij de opstart van de Arduino  
4 }  
5  
6 void loop()  
7 {  
8     // Hier komt code die steeds opnieuw wordt uitgevoerd, na het uitvoeren van setup  
9 }
```

setup()

- Wordt **één keer uitgevoerd**, direct nadat de Arduino wordt aangezet of gereset.
- Gebruik je om de **startinstellingen** te maken.
- Typische voorbeelden:
 - Pinnen instellen als **input** (invoer, bv. een knop) of **output** (uitvoer, bv. een led).
 - Starten van communicatie.

loop()

- Wordt direct na **setup()** uitgevoerd.
- Wordt **steeds opnieuw uitgevoerd**, in een eindeloze lus.
- Hier schrijf je de code die je Arduino continu moet herhalen.
- Typische voorbeelden:
 - Een led aan- en uitzetten.
 - Lezen van een sensorwaarde.
 - Reageren op een drukknop.

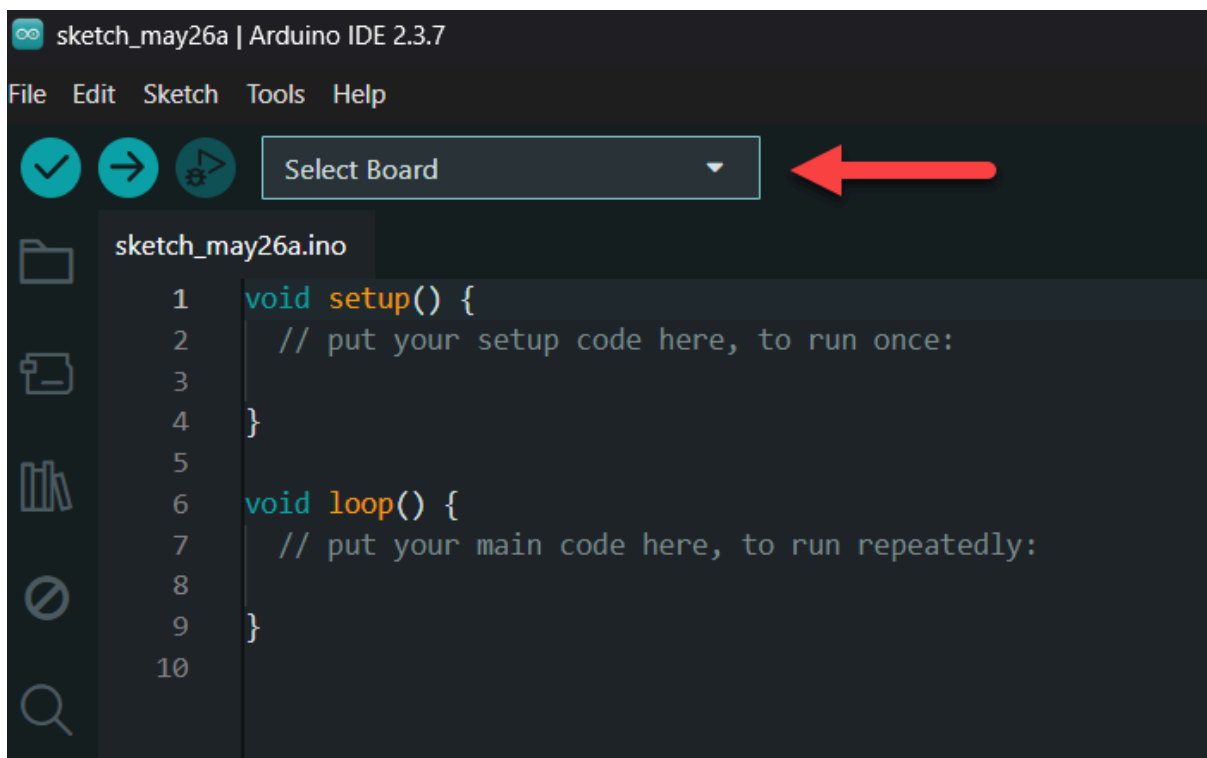
Een programma uploaden

Stap voor stap van een lege Arduino IDE naar een draaiend programma: het juiste board en de juiste port kiezen, het voorbeeldprogramma Blink openen, compileren en uploaden. Op het einde weet je ook hoe je ziet dat de upload wel of niet gelukt is.

We gebruiken hier **Blink** als testprogramma. Dat is het voorbeeld dat de ingebouwde led van je bord laat knipperen. Je hoeft er niets voor aan te sluiten en je ziet aan het bord zelf of de upload geslaagd is, en daarmee is het geschikt om je programmeeromgeving te testen.

1. Je bord en poort kiezen

1. Open de Arduino IDE.
2. Verbind de Arduino via de USB-kabel met je computer.
3. Als je klikt op **Select board**, dan zou je jouw Arduino in de lijst moeten zien.



 De Arduino verschijnt in de lijst onder Select board.

4. Klap die lijst open en kies de lijn met jouw bord. Daar staat niet alleen het type bord (**Arduino Uno / Leonardo**) maar ook de **poort** waarop het aangesloten is.

Afbeelding ontbreekt

TODO: screenshot van de opengeklapte Select board-dropdown met Arduino Uno en de COM-poort zichtbaar

In de opengeklapte lijst zie je zowel het bord als de poort staan.

i WELKE POORT IS DAT?

Op **Windows** heet een poort **COM** gevolgd door een nummer, bijvoorbeeld **COM3**. Op **macOS** en **Linux** is het een apparaatnaam die begint met **/dev/tty** of **/dev/cu.usbmodem**. Welke poort je Arduino krijgt, hangt af van je computer en van de USB-aansluiting waarin je de kabel steekt, dus dat nummer kan veranderen als je een andere poort gebruikt.

Je kan hetzelfde ook nakijken en instellen via **Tools > Port**. Onderaan rechts in het venster van de IDE staat altijd welk bord en welke poort op dit moment geselecteerd zijn.

2. Het voorbeeldprogramma Blink openen

De Arduino IDE opent met een lege sketch, met een leeg **setup()** en een leeg **loop()**. Dat programma doet nog niets, dus open het voorbeeldprogramma via **File > Examples > 01.Basics > Blink**.

Afbeelding ontbreekt

TODO: screenshot van het opengeklapte menu File groter dan Examples groter dan 01.Basics groter dan Blink

Blink zit bij de meegeleverde voorbeelden onder 01.Basics.

Blink opent in een **nieuw venster**, met de naam **Blink** in de titelbalk en **Blink.ino** op het tabblad. In dat venster staat een volledig programma: **setup()** zet de ingebouwde led als uitgang, en **loop()** zet die led afwisselend aan en uit met telkens een seconde ertussen.

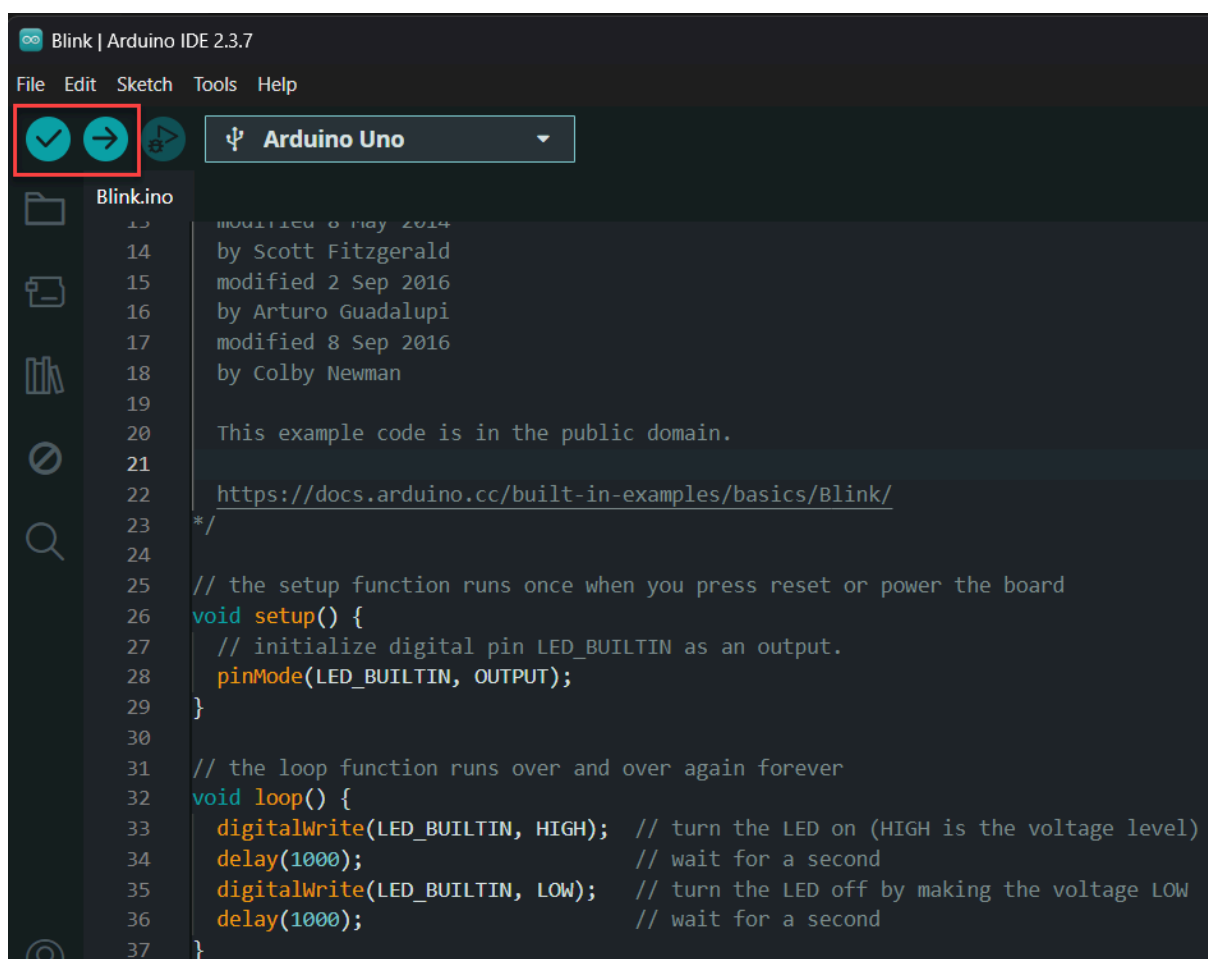
ZELF EEN PROGRAMMA SCHRIJVEN

Voor je eigen programma's begin je met een lege sketch via **File > New Sketch**. Verifiëren en uploaden werkt daarna voor elk programma op dezelfde manier als hieronder.

Pas je een voorbeeld zoals Blink aan, dan kan je het niet zomaar bewaren op zijn oorspronkelijke plaats. De IDE vraagt je dan om het eerst ergens anders op te slaan via **Save As**. Zo blijven de voorbeelden altijd in hun originele vorm staan.

3. Verifiëren

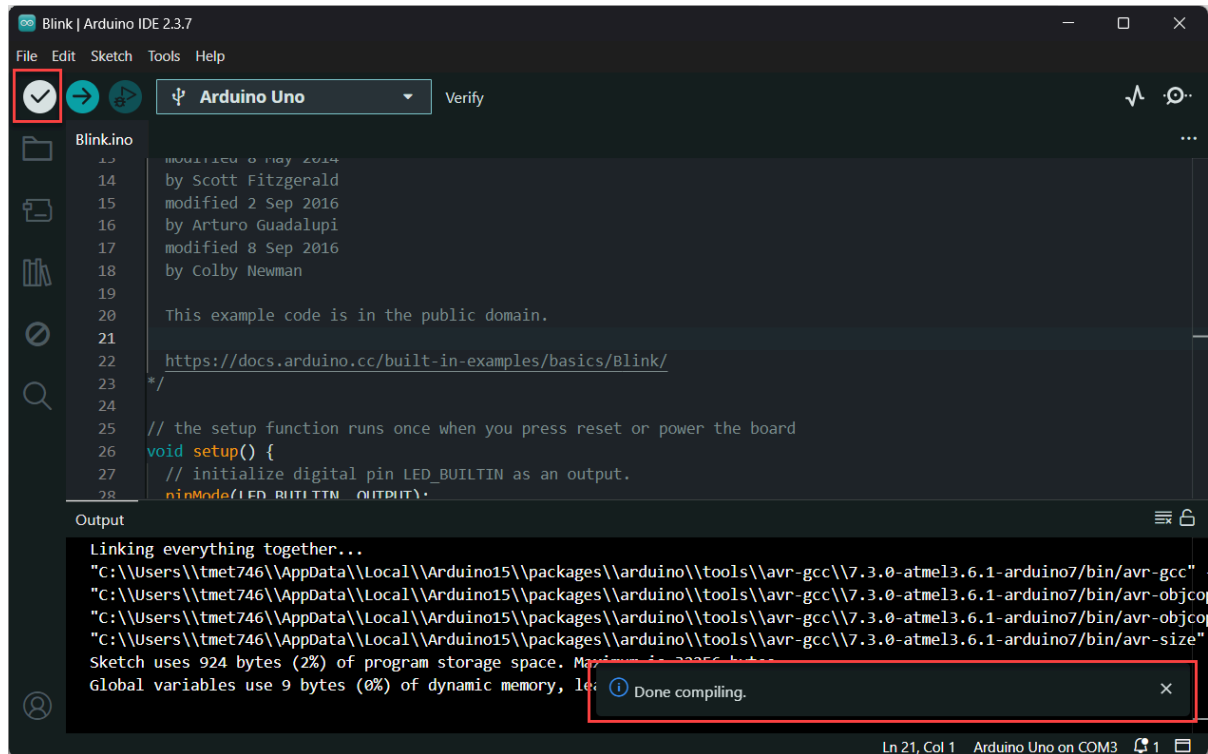
Als je klikt op **Verify**, de knop helemaal links in de werkbalk, dan wordt de code gecompileerd.



 De Verify-knop staat helemaal links in de werkbalk.

Als Verify gelukt is zonder fouten dan krijg je de boodschap **Done compiling**.

Als Verify niet lukt dan krijg je de boodschap **Compilation error** en moet je zelf de syntaxfout zoeken.



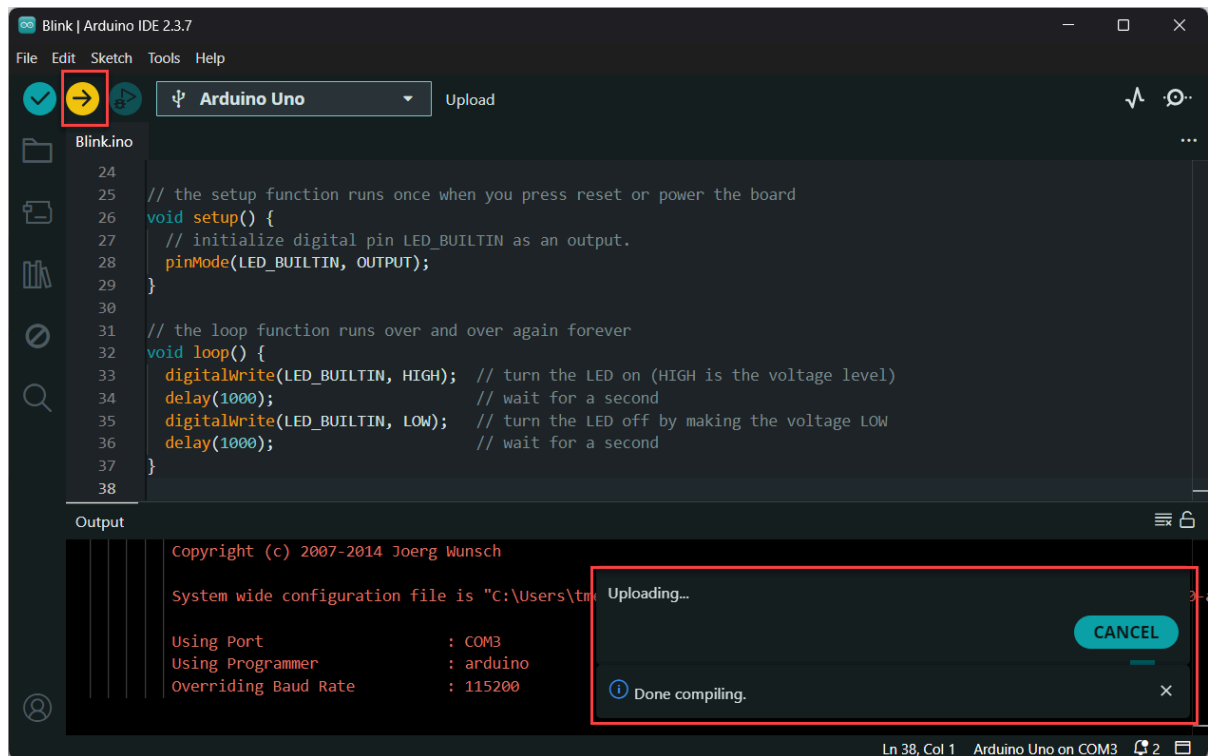
☑ Done compiling: je programma is foutloos vertaald, maar staat nog niet op de Arduino.

4. Uploaden

Meestal ga je de moeite niet doen om op Verify te klikken en wil je gewoon **Uploaden** naar de Arduino om meteen te testen.

De **Upload** knop gaat eerst een **Verify** uitvoeren om zeker te zijn dat er geen syntaxfouten in het programma zitten en de Arduino voldoende geheugen heeft om je programma op te slaan en uit te voeren.

In het **Output window** zie je misschien rode tekst maar dat is niet noodzakelijk een foutmelding.



🔗 Tijdens het uploaden staat er *Uploading* in beeld en knipperen de leds RX en TX op je bord.

5. Hoe weet je dat het gelukt is?

Als de upload klaar is, verschijnt de boodschap **Done uploading**.

Afbeelding ontbreekt

TODO: screenshot van de melding Done uploading in de Arduino IDE

Done uploading: je programma staat nu in het geheugen van de microcontroller.

Vanaf dat moment draait je programma op de Arduino zelf. Kijk naar je bord: de kleine led met de letter L ernaast knippert nu één seconde aan en één seconde uit. Dat blijft zo, ook als je de USB-kabel uittrekt en het bord op een andere manier voedt. Het programma staat namelijk in de microcontroller en niet op je laptop.

Afbeelding ontbreekt

TODO: foto van een Arduino Uno waarop de ingebouwde led L brandt

De ingebouwde led L knippert: Blink draait.

Als het misloopt

- **Je bord staat niet in de lijst.** Probeer eerst een andere USB-kabel. Heel wat kabels die bij een gsm geleverd worden, zijn laadkabels zonder datadraden: daarmee gaat de voedingsled op je Arduino wel branden, maar ziet je computer geen enkel apparaat. Helpt dat niet, probeer dan een andere USB-poort van je laptop.
- **Je krijgt een foutmelding dat de poort bezet is** (op Windows bijvoorbeeld *Access is denied* bij COM3). Dan houdt een ander venster de poort vast. Sluit de seriële monitor, of sluit een tweede venster van de IDE dat nog naar hetzelfde bord verwijst, en probeer opnieuw.
- **Je krijgt Compilation error.** Dan is er nog niets naar de Arduino gestuurd en zit er een fout in de code zelf. Lees de eerste rode lijn in het Output window: daar staan het bestand en het lijnnummer waar de compiler vastliep.

De spelregels van C++

De compiler leest je sketch letterlijk. Eén hoofdletter verkeerd of één puntkomma te weinig en je programma compileert niet. Hieronder staan de vormregels waar C++ op staat, telkens aangewezen op de blink-sketch die je al hebt geüpload.

Dit is de sketch waar de hele pagina naar verwijst. Hij zet de ingebouwde led op pin 13 elke seconde aan en uit.

CPP

```
1  void setup()
2  {
3      pinMode(13, OUTPUT);    // pin 13 wordt een uitgang
4  }
5
6  void loop()
7  {
8      digitalWrite(13, HIGH); // led aan
9      delay(1000);            // een seconde wachten
10     digitalWrite(13, LOW);  // led uit
11     delay(1000);            // een seconde wachten
12 }
```

Wat `setup()` en `loop()` precies doen, staat op [Setup en loop](#). Deze pagina gaat over de vorm van de code, niet over wat ze uitvoert.

De puntkomma

Elke instructie eindigt op een puntkomma. Voor de compiler is dat het teken dat de instructie af is. In de sketch hierboven staat er dus een achter `pinMode(13, OUTPUT);`, achter elke `digitalWrite(...)` en achter elke `delay(1000);`.

Achter een accolade komt er geen puntkomma. `void setup()` met het blok eronder is een functiedefinitie en geen instructie, dus daar sluit de sluitende accolade het geheel af.

Vergeet je een puntkomma, dan leest de compiler jouw regel en de volgende als één instructie. De fout die hij meldt, staat daarom vaak op de regel *onder* de regel waar je de puntkomma vergeten bent.

⚠ EEN PUNKKOMMA ACHTER IF OF WHILE

Een puntkomma op zichzelf is een geldige instructie: een lege. Zet je er een achter de voorwaarde van een `if`, dan is dat de volledige instructie die de `if` uitvoert. Het blok eronder staat er dan los van en wordt altijd uitgevoerd.

```
1  if (aantal == 5);  
2  {  
3    digitalWrite(13, HIGH);  // gebeurt ook als aantal 3 is  
4  }
```

CPP

Bij een `while` wordt die lege instructie de lus zelf. Is de voorwaarde waar, dan herhaalt je Arduino eindeloos niets en blijft je programma op die ene regel hangen.

```
1  while (aantal < 5);  
2  {  
3    aantal++;               // wordt nooit bereikt, aantal blijft staan  
4  }
```

CPP

In beide gevallen is de vorm van je code correct, dus de compiler geeft je geen foutmelding. Je merkt het pas aan wat je programma doet. Zet in **File > Preferences** de **Compiler warnings** op **All**, dan waarschuwt hij je hier wel voor.

Wat `if` en `while` zelf doen, lees je in [Selecties](#) en [Iteraties](#).

De accolades

Een paar accolades `{ }` bundelt instructies tot één blok. In de blink-sketch staan twee blokken: het blok van `setup()` met één instructie erin, en het blok van `loop()` met vier.

Elke openende accolade hoort bij precies één sluitende. Vergeet je er een, dan loopt de compiler verder tot het einde van het bestand voor hij merkt dat er iets mist, en meldt hij de fout dus ergens veel verder dan waar ze staat.

In dit vak zet je de openende accolade op een eigen regel, recht onder de regel waar het blok bij hoort. Zo staat ze in dezelfde kolom als de sluitende accolade die erbij hoort, en zie je in één oogopslag welke twee bij elkaar horen.

Hoofdletters tellen mee

C++ maakt verschil tussen hoofdletters en kleine letters. `digitalWrite` en `digitalwrite` zijn voor de compiler twee verschillende namen, en alleen de eerste bestaat. Schrijf je de tweede, dan meldt de compiler dat hij die naam nergens kent.

Hetzelfde geldt voor de waarden die je meegeeft. `OUTPUT`, `INPUT`, `HIGH` en `LOW` staan volledig in hoofdletters.

En het geldt voor de namen die je zelf verzint. Noem je bovenaan iets `ledPin` en schrijf je verderop `ledpin`, dan zoekt de compiler naar een naam die je nooit gemaakt hebt.

Namen die je zelf kiest

Voor de namen van je eigen variabelen, constanten en functies gelden een paar regels:

- Een naam bestaat uit letters, cijfers en het liggende streepje `_`.
- Een naam begint met een letter, nooit met een cijfer. `pin2` mag, `2pin` mag niet.
- Er staan geen spaties in. `pin led` leest de compiler als twee losse namen.
- Woorden die de taal zelf gebruikt, zijn bezet. `int`, `void`, `if` en `for` kan je niet als naam kiezen.

Daarnaast is er de afspraak van dit vak: je schrijft namen in camelCase, met een kleine letter vooraan en een hoofdletter bij elk volgend woord. Zo wordt de pin van een led `ledPin` en niet `ledpin` of `led_pin`.

Waar je code staat

Instructies staan altijd binnen een functie. Een `digitalWrite(13, HIGH);` die los boven `setup()` zou staan, hoort bij niets en wordt door de compiler geweigerd.

Boven `setup()`, buiten elk blok, staat wat het hele programma nodig heeft: de `#include`-regels waarmee je een bibliotheek binnenhaalt, en de constanten en variabelen die meer dan één functie gebruikt. Wat je binnen een blok maakt, bestaat alleen daar. Dat verschil staat uitgewerkt in [Constanten, variabelen en gegevenstypes](#).

Eén = is toekennen, twee is vergelijken

Met één isgelijktteken zet je een waarde in een variabele. `aantal = 5;` maakt van `aantal` een vijf.

Met twee isgelijktokens stel je een vraag. `aantal == 5` verandert niets en levert `true` of `false` op, naargelang er een vijf in zit.

De verwarring tussen die twee is de klassieke beginnersfout, en de compiler ziet ze meestal niet: `aantal = 5` is op zich geldige code, ze doet alleen iets anders dan je bedoelde. Je komt `==` voor het eerst echt tegen bij [Selecties](#).

Commentaar

Alles achter `//` slaat de compiler over, tot het einde van die regel. Dat is wat er in de blink-sketch achter `// led aan` gebeurt: de tekst is er voor jou, de Arduino ziet ze nooit.

Voor commentaar over meerdere regels gebruik je `/*` om te openen en `*/` om te sluiten:

```
1  /*
2      Blink
3      Labo 0, eerste sketch
4  */
5  digitalWrite(13, HIGH);  // led aan
```

CPP

De skeletcode op [Setup en loop](#) bestaat volledig uit zulke commentaarregels. Er staat dus geen enkele instructie in, en toch compileert ze.

Inspringen en witruimte

Spaties, tabs en lege regels tellen niet mee voor de compiler. De hele blink-sketch op één lange regel zetten levert exact hetzelfde programma op. Voor wie de code leest, maakt het wel uit, en dat ben jij zelf zodra je sketch langer wordt dan een scherm.

De afspraak is: één instructie per regel, en per blok spring je een niveau in. Alles wat tussen de accolades van `loop()` staat, staat dus even ver van de kantlijn.



DE IDE SPRINGT ZELF IN

Met `Ctrl+T` (of `Tools > Auto Format`) laat je de Arduino IDE je hele sketch opnieuw uitlijnen. Loopt de inspringing daarna vreemd, dan is dat meestal het spoor van een accolade die je vergeten bent.

Constanten, variabelen en gegevenstypes

Een variabele is een plaats in het geheugen waar je een waarde bewaart. Met een gegevenstype spreek je af wat voor soort waarde erin past, en met **const** geef je aan dat de waarde nooit meer verandert.

CPP

```
1  const int ledPin = 2;
2  bool isActief = false;
3  double waarde = 10.5;
4
5  void setup()
6  {
7      isActief = true;
8  }
9
10 void loop()
11 {
12     String tekst = "abc";
13     char c = 'a';
14
15     waarde = waarde + 0.5;
16 }
```

In dit voorbeeld zie je hoe we in Arduino werken met **constanten** en **variabelen**.

- **Constanten** (zoals `const int ledPin = 2;`) krijgen één vaste waarde die tijdens het programma niet meer kan veranderen.
- **Variabelen** (zoals `bool isActief = false;` en `double waarde = 10.5;`) kunnen tijdens de uitvoering van het programma wél andere waarden krijgen. In `setup()` wordt `isActief` bijvoorbeeld aangepast naar `true`. In `loop()` wordt `waarde` telkens met `0.5` verhoogd.

Gegevenstypes

Daarnaast zijn er verschillende **gegevenstypes**:

Type	Beschrijving
<code>bool</code>	Kan enkel <code>true</code> of <code>false</code> zijn.
<code>byte</code>	Een geheel getal van 0 tot en met 255.
<code>int</code>	Een geheel getal. Op een Arduino Uno of Leonardo kan een <code>int</code> waarden aannemen van -32768 tot +32767 .
<code>long</code>	Een geheel getal. Op een Arduino Uno of Leonardo kan een <code>long</code> waarden aannemen van ongeveer -2 miljard tot +2 miljard
<code>double</code>	Een kommagetal. Op een Arduino Uno of Leonardo is de kleinste en grootste waarde die je kan bijhouden ongeveer $\pm 3 \times 10^{38}$
<code>float</code>	Op een Arduino Uno of Leonardo hetzelfde als een <code>double</code> .
<code>String</code>	Een stuk tekst. Kan letters, cijfers en speciale tekens bevatten maar je kan er niet mee rekenen.
<code>char</code>	Eén enkel teken, bv. <code>'a'</code> of <code>'@'</code> of <code>'5'</code> .

Globaal en lokaal

Er is ook een verschil tussen **globale** en **lokale** variabelen:

- De variabelen bovenaan (`ledPin`, `isActief`, `waarde`) zijn **globaal** en kunnen overal in het programma gebruikt worden.
- De variabelen die je in `loop()` schrijft (`tekst`, `c`) zijn **lokaal**: ze bestaan enkel binnen die functie en verdwijnen zodra de functie klaar is.

Wiskundige operatoren

Met wiskundige operatoren kan je berekeningen uitvoeren op variabelen en constanten.

In onderstaande voorbeeldcode zie je de meest gebruikte operatoren.

⚠ EEN DELING HANGT AF VAN HET GEGEVENSTYPE

Bij de deling moet je opletten, want het resultaat hangt af van het gegevenstype dat je gebruikt in teller en noemer. Zijn teller én noemer van het type `int`, dan maak je de gehele deling (`quotientInt`). Is teller en/of noemer van het type `double`, dan maak je de deling met kommagetal (`quotientDouble`).

CPP

```
1  int a = 10;
2  int b = 3;
3  int som;
4  int verschil;
5  int product;
6  int quotientInt;
7  int rest;
8  double quotientDouble;
9  double macht;
10 double wortel;
11 void setup()
12 {
13     som = a + b;           // optellen → 10 + 3 = 13
14     verschil = a - b;      // aftrekken → 10 - 3 = 7
15     product = a * b;      // vermenigvuldigen → 10 * 3 = 30
16     quotientInt = a / b;   // geheel getal delen → 10 / 3 = 3
17     rest = a % b;         // modulo → 10 % 3 = 1
18     quotientDouble = (double) a / b; // delen met kommagetal → 10.0 / 3 = 3.33
19     macht = pow(a, b);     // macht → 10³ = 1000
20     wortel = sqrt(a);     // vierkantswortel → √10 ≈ 3.16
21 }
22 void loop()
23 {
24
25 }
```

Selecties

Een keuze maken in je programma doe je op basis van een voorwaarde. Die voorwaarde stel je op als een vergelijking (is gelijk aan, is groter dan, ...) en die vergelijking moet altijd te herleiden zijn tot **true** of **false**.

De instructie `digitalRead(pinNummer)` leest de status in van een pin. Die kan 1 (= **HIGH**) of 0 (= **LOW**) zijn.

WAAROP TEST JE BIJ EEN KNOP?

De voorbeelden hieronder testen op **HIGH**, wat past bij een knop met een **pull-down**weerstand. Werk je met een **pull-up** of met `INPUT_PULLUP`, dan is een ingedrukte knop juist **LOW** en test je dus op **LOW**. Zie [Pull up en pull down weerstanden](#). De selectiestructuur zelf blijft in beide gevallen dezelfde.

Voorbeeld enkelvoudige selectie

CPP

```
1  const int knopPin = 3;
2
3  void setup()
4  {
5      pinMode(knopPin, INPUT);
6  }
7
8  void loop()
9  {
10     int waarde = digitalRead(knopPin);
11     if(waarde == HIGH) // de voorwaarde is te herleiden tot true / false
12     {
13         // doe iets als de voorwaarde true is
14     }
15 }
```

Voorbeeld tweevoudige selectie

CPP

```
1  const int knopPin = 3;
2
3  void setup()
4  {
5      pinMode(knopPin, INPUT);
6  }
7
8  void loop()
9  {
10     int waarde = digitalRead(knopPin);
11     if(waarde == HIGH) // de voorwaarde is te herleiden tot true / false
12     {
13         // doe iets als de voorwaarde true is
14     }
15     else
16     {
17         // doe iets anders als de voorwaarde false is
18     }
19 }
```

Een tweevoudige selectie in het kort

Kies je met een `if/else` tussen twee **waarden**, en verder niets, dan mag dat ook in één regel. Daarvoor bestaat de operator `?:`. Je schrijft eerst de voorwaarde, dan een vraagteken, dan de waarde als de voorwaarde waar is, dan een dubbele punt, en tot slot de waarde als ze niet waar is:

```
voorwaarde ? waardeAlsWaar : waardeAlsNietWaar
```

CPP

Deze twee stukken code doen exact hetzelfde:

CPP

```
1 // De volledige schrijfwijze
2 if (waarde == HIGH)
3 {
4     digitalWrite(ledPin, HIGH);
5 }
6 else
7 {
8     digitalWrite(ledPin, LOW);
9 }
10
11 // Dezelfde keuze, in het kort
12 digitalWrite(ledPin, waarde == HIGH ? HIGH : LOW);
```

Het verschil met een gewone `if` is dat `?:` zelf een **waarde oplevert**. Je kan hem daarom rechtstreeks meegeven als argument van een functie, zoals hierboven, of in een toekenning zetten:

CPP

```
int helderheid = knopIngedrukt ? 255 : 0;
```

Voorbeeld meervoudige selectie

CPP

```
1 const int sensorPin = A0;
2
3 void setup()
4 {
5     pinMode(sensorPin, INPUT);
6 }
7
8 void loop()
9 {
10     int waarde = analogRead(sensorPin);
11     if(waarde > 768)
12     {
13
14     }
15     else if (waarde > 512)
16     {
17
18     }
19     else
20     {
21
22     }
23 }
```

WAT `ANALOGREAD()` TERUGGEEFT

De instructie `analogRead(pinNummer)` leest de analoge status (= de spanning) in van een pin. Dit kan een waarde tussen 0 en 1023 zijn waarbij 0 overeenkomt met 0 V en 1023 overeenkomt met 5 V.

Voorbeeld meervoudige selectie met logische operatoren

CPP

```
1  const int knopPin = 3;
2  const int sensorPin = A0;
3
4  void setup()
5  {
6      pinMode(knopPin, INPUT);
7      pinMode(sensorPin, INPUT);
8  }
9
10 void loop()
11 {
12     int waarde = analogRead(sensorPin);
13     int status = digitalRead(knopPin);
14
15     if(waarde > 768 && status == HIGH)
16     {
17
18     }
19     else if (waarde > 512 || status == LOW)
20     {
21
22     }
23     else
24     {
25
26     }
27 }
```

Comparator	Betekenis
>	Groter dan
>=	Groter dan of gelijk aan
<	Kleiner dan
<=	Kleiner dan of gelijk aan
==	Gelijk aan
!=	Verschillend van

Operator	Betekenis
&&	AND
	OR
!	NOT

Iteraties

Naast de sequentie en de selectie is de iteratie één van de drie basisstructuren waarmee een programma wordt opgebouwd. Itereren betekent letterlijk herhalen. Het is dus een structuur om een stuk code een aantal keer te herhalen. Dit kan een oneindige herhaling zijn zoals de `loop()`-functie, maar kan ook een conditionele herhaling zijn zoals een `for`-lus of een `while`-constructie.

De for-instructie

Bij de `for`-instructie gaat het om een lus waarvan we op voorhand weten hoeveel keer hij moet doorlopen worden. Dit aantal keer kan ook een variabele waarde zijn, maar het gaat erom dat het zeker is dat na x aantal keer de lus te hebben doorlopen het programma verder gezet wordt.

Syntax

```
1  for(initialisatie; voorwaarde; aanpassing)
2  {
3      // instructies
4  }
```

CPP

Parameters

- **initialisatie:** gebeurt eerst en eenmalig. Hier wordt een variabele op een waarde geïnitieerd. Deze variabele gaat via de voorwaarde bepalen of de lus nog verder doorlopen wordt of beëindigd wordt. Typisch gebruik je hiervoor een tellervariabele die je in de initialisatie zelf definieert.
- **voorwaarde:** een uitdrukking die resulteert in een booleaanse waarde `true` of `false`. Indien de voorwaarde `true` is, zal de code binnen de lus opnieuw doorlopen worden. Bij een `false` wordt de lus stopgezet en gaan we verder met de eerste instructie na de lusconstructie.
- **aanpassing:** na elke doorloop van de lus wordt deze instructie uitgevoerd. In de eenvoudigste vorm wordt hier een teller, die gedefinieerd werd bij de initialisatie, met een waarde verhoogd of verlaagd, waardoor deze variabele een nieuwe waarde krijgt. Deze wordt dan afgetoetst met de voorwaarde om te bepalen of de lus nog verder moet doorlopen worden.

Voorbeeld

CPP

```
1  for(int i=0;i<10;i++)
2  {
3      Serial.println(i);
4  }
```

In bovenstaand voorbeeld wordt een variabele `i` van het type integer op 0 geïnitieerd. Daarna wordt de voorwaarde een eerste maal geëvalueerd, waarbij het resultaat `true` is, want `0<10` resulteert in `true`. De luscode wordt een eerste keer doorlopen en via het serieel kanaal zien we 0 op de terminal verschijnen.

Na het uitvoeren van de luscode wordt de aanpassing uitgevoerd, en krijgt `i` de nieuwe waarde 1 door de verhoging ten gevolge van `i++`.

De voorwaarde wordt opnieuw geëvalueerd, waardoor er opnieuw een `true` als resultaat gevonden wordt: `1<10` is immers waar. De luscode wordt dus een tweede maal doorlopen en op de terminal verschijnt een 1.

Opnieuw wordt `i` met één verhoogd, waardoor `i` gelijk wordt aan twee. We kunnen deze redenering voortzetten en zullen zien dat op de terminal achtereenvolgens nog de 2, 3, 4, 5, 6, 7, 8 en 9 verschijnen.

Bij `i=9` zal de aanpassing ervoor zorgen dat `i` nu de waarde 10 krijgt. De voorwaarde `10<10` geeft nu als resultaat `false`, waardoor de lus niet meer uitgevoerd wordt en het programma verder gaat met de eerste instructie na de lus.

⚠ EEN BYTE LOOPT MAAR TOT 255

Bekijk dit programma:

```
1  void setup()
2  {
3      Serial.begin(9600);
4  }
5  void loop()
6  {
7      for(byte i=250;i<260;i++)
8      {
9          Serial.println(i);
10     }
11 }
```

CPP

Op het eerste gezicht zou je denken dat de lus 10 keer doorlopen wordt, met de waarden voor `i` die variëren van 250 tot en met 259, maar in de praktijk is dit niet waar.

Doordat `i` als `byte` gedefinieerd wordt, loopt zijn bereik slechts tot en met 255. Daarna krijg je een overflow en wordt `i` gelijk aan 0. De waarde 260 wordt dus nooit bereikt, met als gevolg dat deze lus oneindig gaat blijven doorlopen.

Hierboven hebben we de standaardwerking van de for-lus bekeken, maar er zijn nog heel veel andere mogelijkheden. Stel dat je de eerste 10 machten van 2 wil afprinten op de terminal, dan kan dit op verschillende manieren aangepakt worden. Je kan een variabele gebruiken om de waarde elke keer te verdubbelen. De code ziet er dan als volgt uit:

```
1  void setup()
2  {
3      Serial.begin(9600);
4  }
5
6  void loop()
7  {
8      int macht=1;
9      for(byte i=0;i<10;i++)
10     {
11         Serial.println(macht);
12         macht=macht*2;
13     }
14 }
```

CPP

Een tweede manier is om de waarde van de macht te berekenen met behulp van een functie. In C++ bestaat daarvoor de functie `pow(grondtal, exponent)`. Het resultaat is een float.

```
1  void setup()
2  {
3      Serial.begin(9600);
4  }
5
6  void loop()
7  {
8      for(byte i=0;i<10;i++)
9      {
10         Serial.println(pow(2,i));
11     }
12 }
```

CPP

Een derde manier maakt gebruik van de lus zelf. We laten de teller niet gewoon optellen, maar we verdubbelen hem elke keer hij door de lus geweest is.

```
1  void setup()
2  {
3      Serial.begin(9600);
4  }
5
6  void loop()
7  {
8      for(int i=1;i<513;i=i*2)
9      {
10         Serial.println(i);
11     }
12 }
```

CPP

De while-structuur

Een tweede iteratiestructuur is de while-instructie. Hier gaat het erom dat je op voorhand niet weet hoe lang de lus moet doorlopen worden. Bij de while-structuur is het zelfs helemaal niet zeker of de luscode ooit moet doorlopen worden. Alles is afhankelijk van de voorwaarde.

Syntax

```
1  while (voorwaarde)
2  {
3      // instructies
4  }
```

CPP

Parameters

voorwaarde: expressie die in **true** of **false** resulteert.

Voorbeeld

```
1  void setup()
2  {
3      pinMode(3, INPUT_PULLUP);
4  }
5  void loop()
6  {
7      while(digitalRead(3))
8      {
9      }
10 }
```

CPP

In bovenstaand voorbeeld wordt de luscode (waar niks in staat) uitgevoerd zolang er een hoog niveau op pin 3 staat. Zodra daar een laag niveau op komt, gaat het programma verder. Zo'n constructie kan gebruikt worden als je bijvoorbeeld wil dat het programma niet start voordat er op een startknop wordt gedrukt. Deze startknop kan dan tussen GND en pin 3 geplaatst worden.

⚠ GEVAAR: ONEINDIGE LUS

Aangezien de luscode uitgevoerd wordt zolang de voorwaarde resulteert in `true`, moet je extra opletten bij het opstellen van de voorwaarde.

Bijvoorbeeld:

```
1 void setup()
2 {
3   Serial.begin(9600);
4 }
5
6 void loop()
7 {
8   bool voorwaarde = false;
9   while(voorwaarde=true)
10  {
11    Serial.print("surprise");
12  }
13 }
```

CPP

In bovenstaande code verwacht je op het eerste gezicht dat de tekst "surprise" nooit op de terminal zal verschijnen. In werkelijkheid hebben we een fout gemaakt bij het opstellen van de voorwaarde, omdat we een enkel `=`-teken gebruikt hebben i.p.v. een dubbel `=`-teken. Daardoor doen we hier een toekenning van `true` aan de variabele "voorwaarde", waardoor de uitdrukking tussen de haakjes van de while-functie resulteert in `true`, en de tekst dus toch verschijnt op de terminal.

Merk ook op dat je bij een while-constructie er altijd voor moet zorgen dat in de voorwaarde of in het while-codeblok een instructie zit die ervoor kan zorgen dat de voorwaarde verandert van logische toestand. Als dit niet het geval is, krijgen we een oneindige lus en zit het programma vast in deze oneindige lus.

i EEN PROGRAMMA PERMANENT STOPPEN

Het kan soms de bedoeling zijn om het programma permanent te stoppen. Dat lijkt onmogelijk, aangezien het programma binnen de `loop()`-functie draait die oneindig herhaalt.

Je kan met de while-instructie echter zorgen voor een oneindige lus waardoor de uitvoering van het programma gestopt lijkt voor de buitenwereld. Gebruik daarvoor de constructie:

```
1 while(true)
2 {
3 }
```

CPP

De do-while-structuur

Een variant op de while-structuur is de do-while-structuur. Hierbij komt de voorwaarde om de lus verder uit te voeren op het einde van de luscode, waardoor de luscode gegarandeerd één keer uitgevoerd wordt.

Syntax

```
1  do
2  {
3      // instructies
4  } while (voorwaarde);
```

CPP

Parameters

voorwaarde: idem zoals bij de while-structuur.

Functies en parameters

Een **functie** is een stuk code dat een bepaalde taak uitvoert. Het voordeel is dat je die taak kan **hergebruiken** zonder telkens opnieuw dezelfde code te schrijven.

Elk Arduino-programma heeft standaard twee functies:

- **setup()**: dit wordt **één keer uitgevoerd** wanneer de Arduino opstart. Hier zet je meestal de begininstellingen (bv. pinnummers instellen).
- **loop()**: dit wordt **telkens opnieuw herhaald** zolang de Arduino aanstaat. Hier zet je de code die continu moet draaien.

Eigen functies maken

Je kan ook zelf functies maken. Bijvoorbeeld:

CPP

```
1  const int ledPin = 2;
2
3  void setup()
4  {
5      pinMode(ledPin, OUTPUT);
6  }
7
8  void loop()
9  {
10     knipperLed(200); // laat de LED 200 ms branden
11     knipperLed(1000); // laat de LED 1000 ms branden
12 }
13
14 // functie met parameter
15 void knipperLed(int tijd)
16 {
17     digitalWrite(ledPin, HIGH); // LED aan
18     delay(tijd); // wacht de opgegeven tijd
19     digitalWrite(ledPin, LOW); // LED uit
20     delay(tijd); // wacht de opgegeven tijd
21 }
```

Uitleg

- De functie heet `knipperLed`.
- Tussen de haakjes staat `int tijd`, dat is de **parameter**.
- Wanneer je de functie aanroept (`knipperLed(200)` of `knipperLed(1000)`), vul je die parameter in met een waarde.
- Binnen de functie wordt die waarde gebruikt door `delay(tijd)`.

Zo kan je dezelfde functie gebruiken voor **verschillende situaties**, zonder dat je code moet kopiëren of herschrijven.

pinMode()

Een pin van de Arduino kan als **input of output** gebruikt worden, en met `pinMode()` leg je vast welke van de twee het wordt.

Hoe definieer je een pin?

Vooraleer je een pin kan gebruiken om iets aan te sturen of iets in te lezen, moet je deze pin voor die functie instellen. Dit gebeurt met behulp van de `pinMode`-instructie.

Syntax

```
pinMode(pin, mode);
```

CPP

Parameters

- **pin**: nummer van de pin die je wil instellen.
- **mode**: `INPUT`, `OUTPUT` of `INPUT_PULLUP`.

Geeft terug

Niets

Voorbeeld

In een programma ziet dat er dan bijvoorbeeld zo uit:


```
1  const int ledPin = 2;
2  const int knopPin = 3;
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT); // een led stuur je aan: uitgang
7      pinMode(knopPin, INPUT); // een knop lees je uit: ingang
8  }
9
10 void loop()
11 {
12     // Hier komt code die steeds opnieuw wordt uitgevoerd
13 }
```

Wat is het verschil?

INPUT

Standaard staat elke pin in INPUT-mode. Dit is een [hoogimpedante](#) instelling waarbij de ingangsweerstand van de pin zo'n 100 MOhm is. Dit betekent dat er zeer weinig stroom nodig is om de pin van een lage naar een hoge toestand te brengen. Daarmee is deze pin dan ook gevoelig voor storing uit de nabijheid wanneer er niets op aangesloten is. Zelfs een naburige pin die schakelt, kan er dan voor zorgen dat de toestand van de pin verandert.

Dit kan opgelost worden door extern een pullup- of pulldownweerstand aan te schakelen. Meestal gebruik je daarvoor een weerstand van 10 KOhm.

INPUT_PULLUP

In deze mode schakel je de ingebouwde pullupweerstand in. Deze ligt tussen 20 KOhm en 50 KOhm en zorgt ervoor dat er, wanneer er niets met de pin verbonden is, altijd een hoge waarde gelezen wordt. Om een laag niveau te lezen, moeten we de pin dus verbinden met de massa.

De meest gebruikte manier hiervoor is een switch, waarvan de NO-kant verbonden wordt met de pin en de COM-kant met de massa. Wanneer de switch niet bediend is, staat op de ingang een hoog niveau vanwege de pullupweerstand. Zodra de schakelaar ingedrukt wordt, krijgen we een verbinding met massa en bijgevolg een laag niveau op de pin. De pin is dus laag assertief.

i INGEBOUWDE LED

Pin 13 is standaard verbonden met de ingebouwde led en is dus minder geschikt om als ingang gebruikt te worden. De led staat dan in serie geschakeld met de pullupweerstand, waardoor een spanning van ongeveer 1,7 V op de ingang staat en deze altijd als LOW wordt gelezen. Wil je pin 13 absoluut als ingang gebruiken, gebruik dan beter de INPUT-mode en voorzie een externe pulldownweerstand. Deze ingang is dan hoog assertief.

OUTPUT

In deze mode staat de pin in [laagimpedante](#) toestand en levert hij stroom aan de buitenwereld. Het **absolute maximum** is 40 mA, maar in de praktijk hou je **20 mA per pin** aan als veilige waarde. Dat is genoeg om een led aan te sturen, maar ruim onvoldoende om een motor aan te sturen. Daarvoor is dan een interfaceschakeling nodig (zie verder). De volledige uitleg over die grenzen staat op [Sourcen en sinken](#).

Hoewel een led rechtstreeks op de pin kan worden aangesloten, is het toch altijd verstandig om een outputpin te voorzien van een externe serieweerstand. Een outputpin die bijvoorbeeld kortgesloten wordt, kan beschadigd worden waardoor hij niet meer werkt. Dit heeft meestal geen gevolgen voor de rest van de controller, maar je houdt dan wel een "dode pin" over waar je niets meer mee kan aanvangen. Voor een led op 5 V kom je uit bij **220 Ohm**, en voor andere toepassingen is 1 KOhm een gangbare waarde. Hoe je die 220 Ohm zelf berekent, lees je op [De wet van Ohm](#).

i STROOMBEPERKING

Je zou verkeerdelijk kunnen rekenen dat we, als elke pin 40 mA kan leveren, in totaal $13 \times 40 \text{ mA} = 520 \text{ mA}$ kunnen leveren. Dit is onmogelijk, aangezien de USB-aansluiting zelf al beperkt is tot 500 mA max. Maar zelfs 500 mA is niet haalbaar omdat **de Arduino de stroom op alle pinnen samen beperkt tot een maximum van 200 mA**.

De voedingspinnen +5V en GND kunnen tot maximum 500 mA leveren via USB, maar indien de Arduino extern gevoed wordt via de barrelconnector door een voeding met voldoende vermogen, dan kunnen we hier tot 1 A gaan. De ingebouwde spanningsregelaar gaat daarna wel in thermal shutdown.

Er is ook een 3,3V-spanningsregelaar aanwezig, die op de 3,3V-pin een maximale stroom van 150 mA kan leveren.

digitalRead / digitalWrite

Met `digitalRead()` en `digitalWrite()` kan je de **digitale ingangen en uitgangen** van de Arduino bedienen:

- `digitalWrite(pin, waarde)` zet een pin **hoog (HIGH)** of **laag (LOW)**, bijvoorbeeld om een led aan of uit te zetten.
- `digitalRead(pin)` leest de toestand van een digitale pin: die staat **hoog (HIGH)** of **laag (LOW)**, bijvoorbeeld naargelang een knop ingedrukt is of niet.

⚠ OF EEN INGEDRUKTE KNOP HIGH OF LOW GEEFT, HANGT VAN JE BEDRADING AF

Dat is geen eigenschap van `digitalRead()` zelf, maar van hoe je de knop aansluit. Bij een **pull-down** weerstand, zoals in het voorbeeld hieronder, lees je in rust een **LOW** en bij het indrukken een **HIGH**. Bij een **pull-up** weerstand of bij `INPUT_PULLUP` is het net omgekeerd: in rust **HIGH**, ingedrukt **LOW**. Die tweede vorm kom je in dit vak het vaakst tegen. Zie [Pull up en pull down weerstanden](#), en let er dus altijd op waarop je test.

CPP

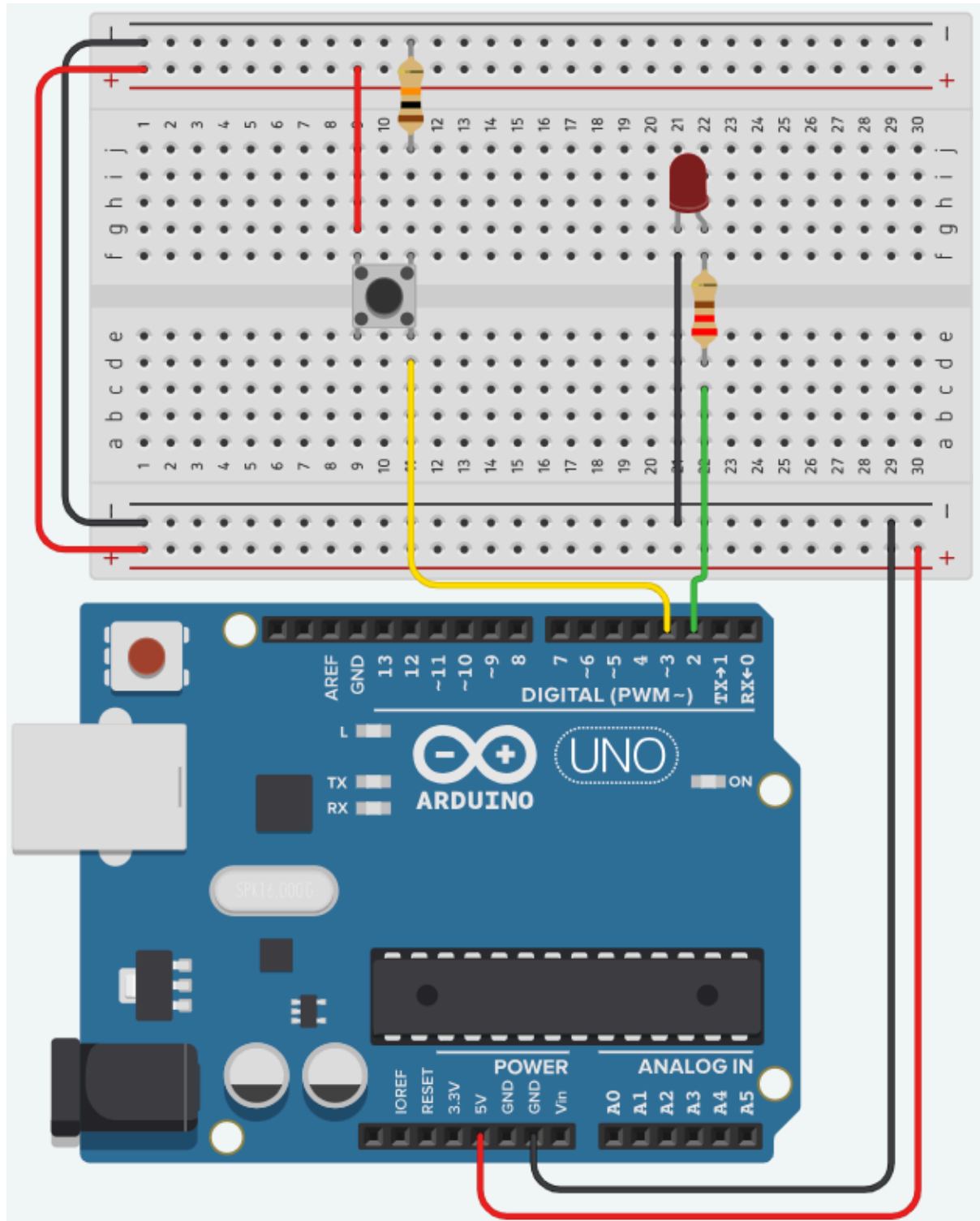
```
1  const int ledPin = 2;
2  const int knopPin = 3;
3
4  void setup()
5  {
6      pinMode(ledPin, OUTPUT); // LED als uitgang
7      pinMode(knopPin, INPUT); // knop als ingang, met externe pull-downweerstand
8  }
9
10 void loop()
11 {
12     int knopStatus = digitalRead(knopPin); // lees de knop
13
14     if (knopStatus == HIGH)
15     {
16         digitalWrite(ledPin, HIGH); // LED aan als knop ingedrukt
17     }
18     else
19     {
20         digitalWrite(ledPin, LOW); // LED uit als knop niet ingedrukt
21     }
22 }
```

- `ledPin` en `knopPin` zijn **globale constanten**, zodat het pinnummer overal hetzelfde is.
- In `setup()` worden de pinnen ingesteld.
- In `loop()` leest de Arduino continu de knop en schakelt de led aan of uit.

💡 HIGH IS 1 EN LOW IS 0

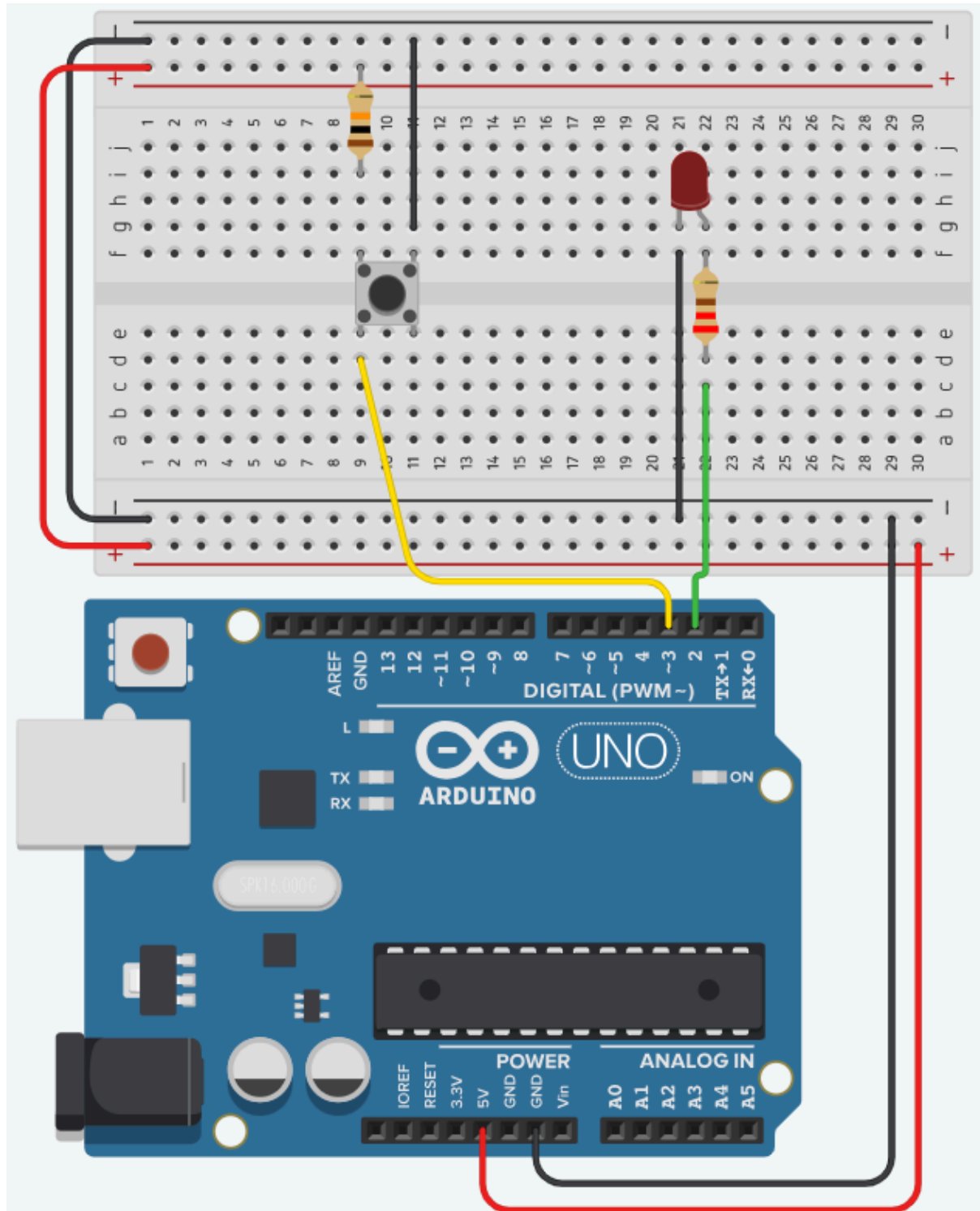
Die twee namen staan in de Arduino-omgeving gewoon voor een getal: `HIGH` is 1 en `LOW` is 0. Daardoor doen `digitalWrite(ledPin, 1)` en `digitalWrite(ledPin, true)` hetzelfde als `digitalWrite(ledPin, HIGH)`, en test `digitalRead(knopPin) == 1` hetzelfde als `digitalRead(knopPin) == HIGH`. `digitalWrite()` kijkt zelfs alleen of de waarde 0 is: elk ander getal zet de pin hoog, zonder foutmelding. Schrijf toch `HIGH` en `LOW`. Die namen maken duidelijk dat het om een spanningsniveau gaat.

Schema met pull-downweerstand



[Aansluitschema met een pull-downweerstand](#)

Schema met pull-upweerstand

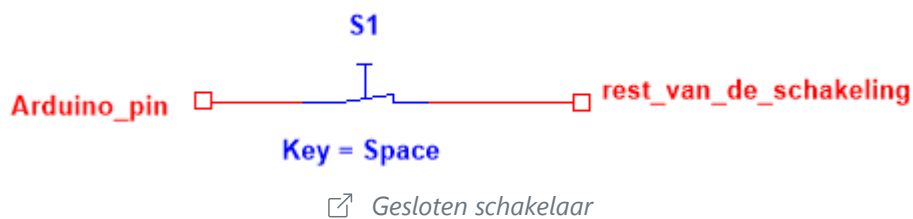
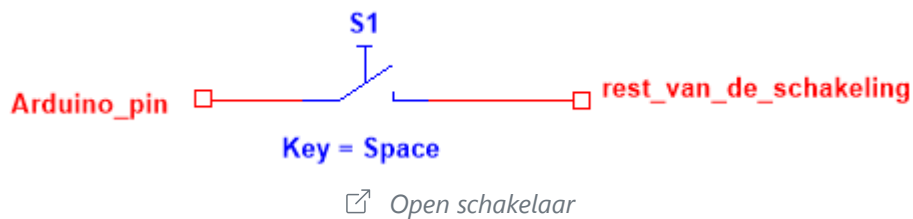


[↗](#) Aansluitschema met een pull-upweerstand

Pull up en pull down weerstanden

Een knop die je op een ingang aansluit, heeft een **pull-up**- of een **pull-down**weerstand nodig om die ingang in rust een vaste waarde te geven. Deze pagina gaat dieper in op het uitlezen van knoppen.

Wat is een knop?

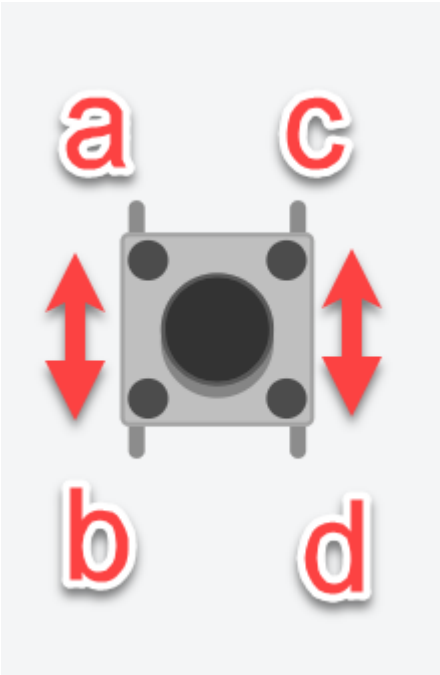
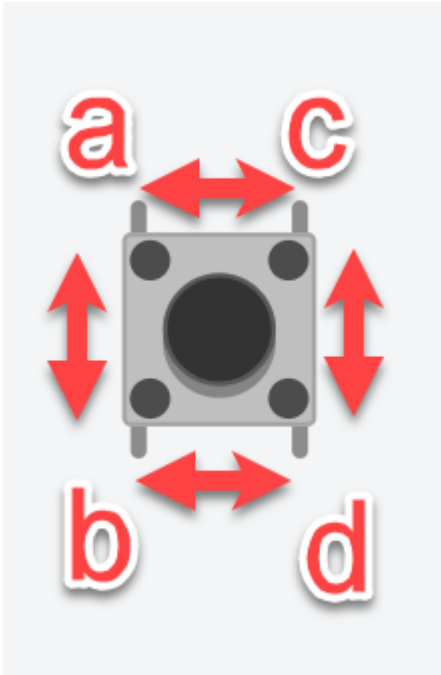


Een knop is niets anders dan een verbinding die je maakt of verbreekt. Knoppen komen dan ook in twee versies voor: de NO en de NC. Deze letterwoorden staan voor "normally open" en "normally closed" en geven aan wat de rusttoestand van de knop is.

Bij een NO is de verbinding standaard verbroken (open) en zorgt het drukken op de knop voor het leggen van de verbinding, terwijl bij een NC de verbinding standaard gelegd is en het drukken op de knop de verbinding verbreekt.

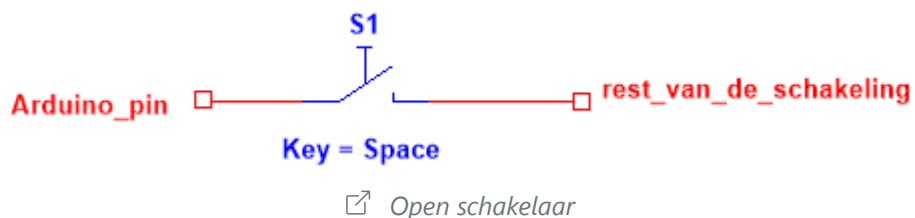
Pushbutton

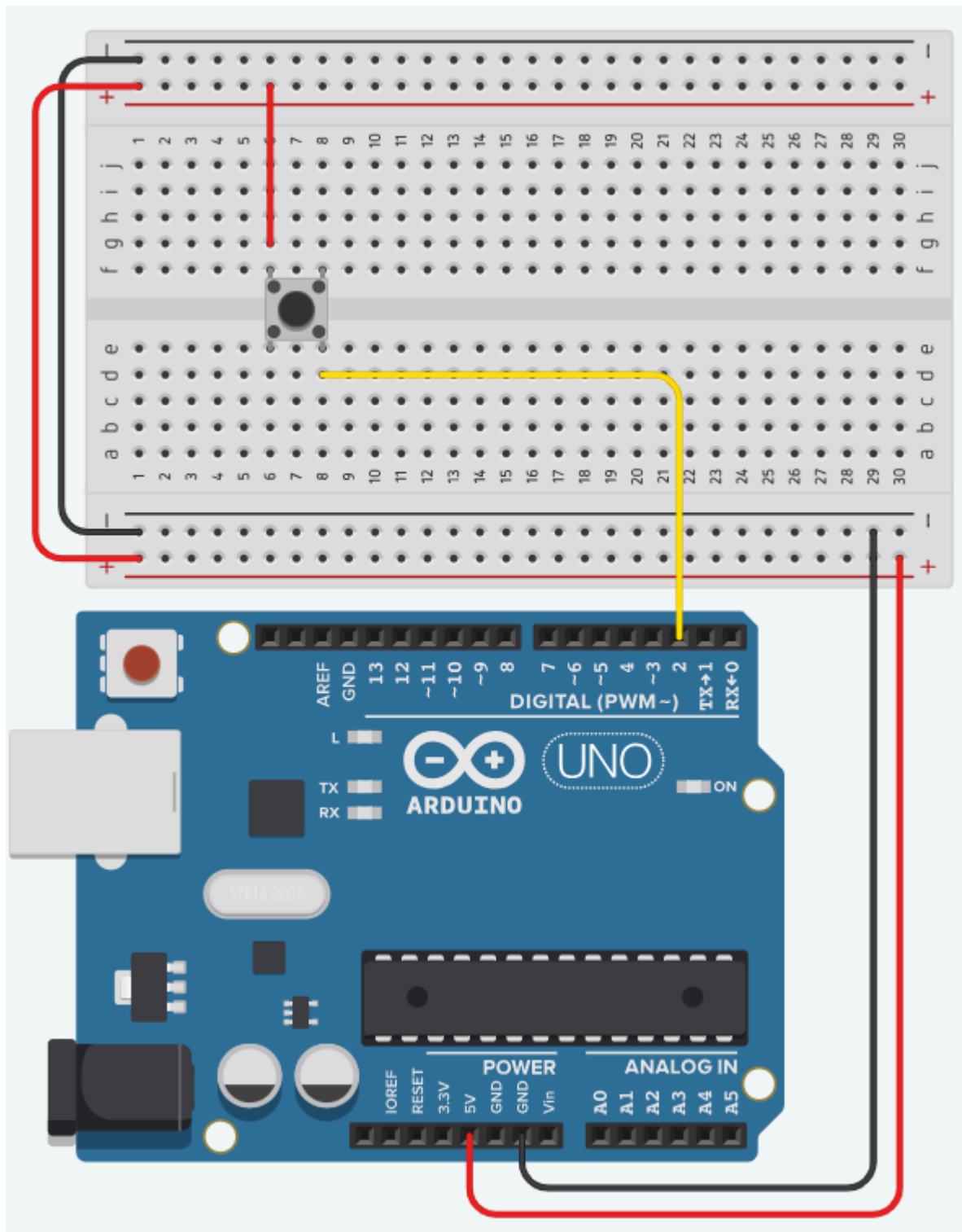
Een gewone schakelaar is bistabiel. Dit wil zeggen dat de status behouden blijft. In onze projecten gaan we echter heel vaak een pushbutton gebruiken. Deze eenvoudige knop gedraagt zich zoals beschreven in de tabel.

Niet ingedrukt	Wel ingedrukt
	
a is verbonden met b maar niet met c of d c is verbonden met d maar niet met a of b	a, b, c en d zijn met elkaar verbonden

Normally open

Er bestaan dus knoppen die in rust "normally open" zijn en waarbij de verbinding standaard onderbroken is. Wanneer we deze knop bekrachtigen, gaan we de verbinding maken. Belangrijk hierbij is om te beseffen dat er, wanneer de knop in rust is, geen verbinding is. Wanneer we zo'n knop verbinden met een ingangspin van de Arduino, hangt er aan de ingang van de Arduino een stuk draad dat met niks verbonden is.





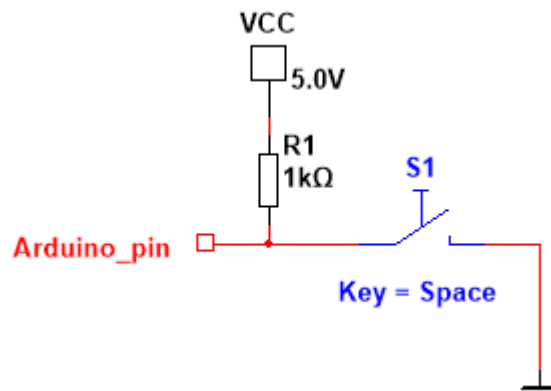
[Klik op de afbeelding om te vergroten](#)

Het probleem is dat zo'n stuk losse draad (in dit geval de gele draad) zich als een antenne gedraagt en alle soorten elektromagnetische storingen uit de omgeving kan oppikken, waardoor de ingang willekeurig een hoog of een laag signaal oppikt.

Een oplossing is om op de ingang te zorgen voor een standaardwaarde. Dit kan ook weer op twee manieren. We kunnen een pull-up- of een pull-downweerstand gebruiken. Daarbij verbind je de ingang via een grote weerstand met massa (pull-down, dus trek de ingang omlaag naar 0 V) of met de voedingsspanning V_{cc} (pull-up, trek de ingang omhoog naar +5 V).

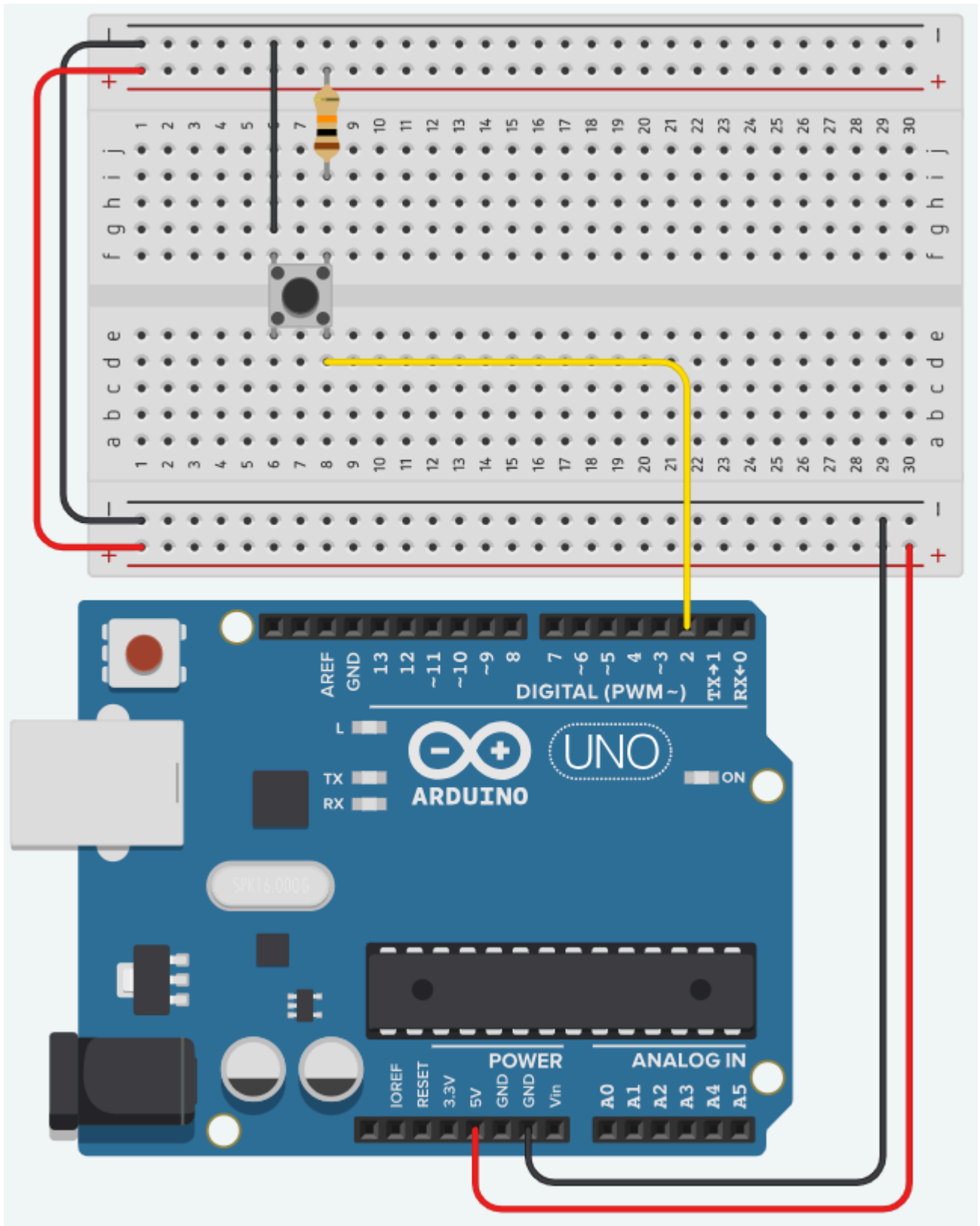
Pull-up

Wanneer we een weerstand van een redelijk grote waarde (meestal 1 k Ω , kan ook 10 k Ω zijn) plaatsen tussen +5 V en de ingang, dan zal deze ingang altijd een 1 lezen wanneer er verder niks aan verbonden is.



 *Ingang met pull-upweerstand*

Als we aan deze ingang ook nog een NO-knop hangen, heeft deze in rust geen invloed en blijven we een 1 lezen. Als we de andere kant van de knop verbinden met de massa, dan zal het indrukken van de knop ervoor zorgen dat de ingang nu rechtstreeks verbonden is met de massa, waardoor we op de ingang een 0 gaan lezen.

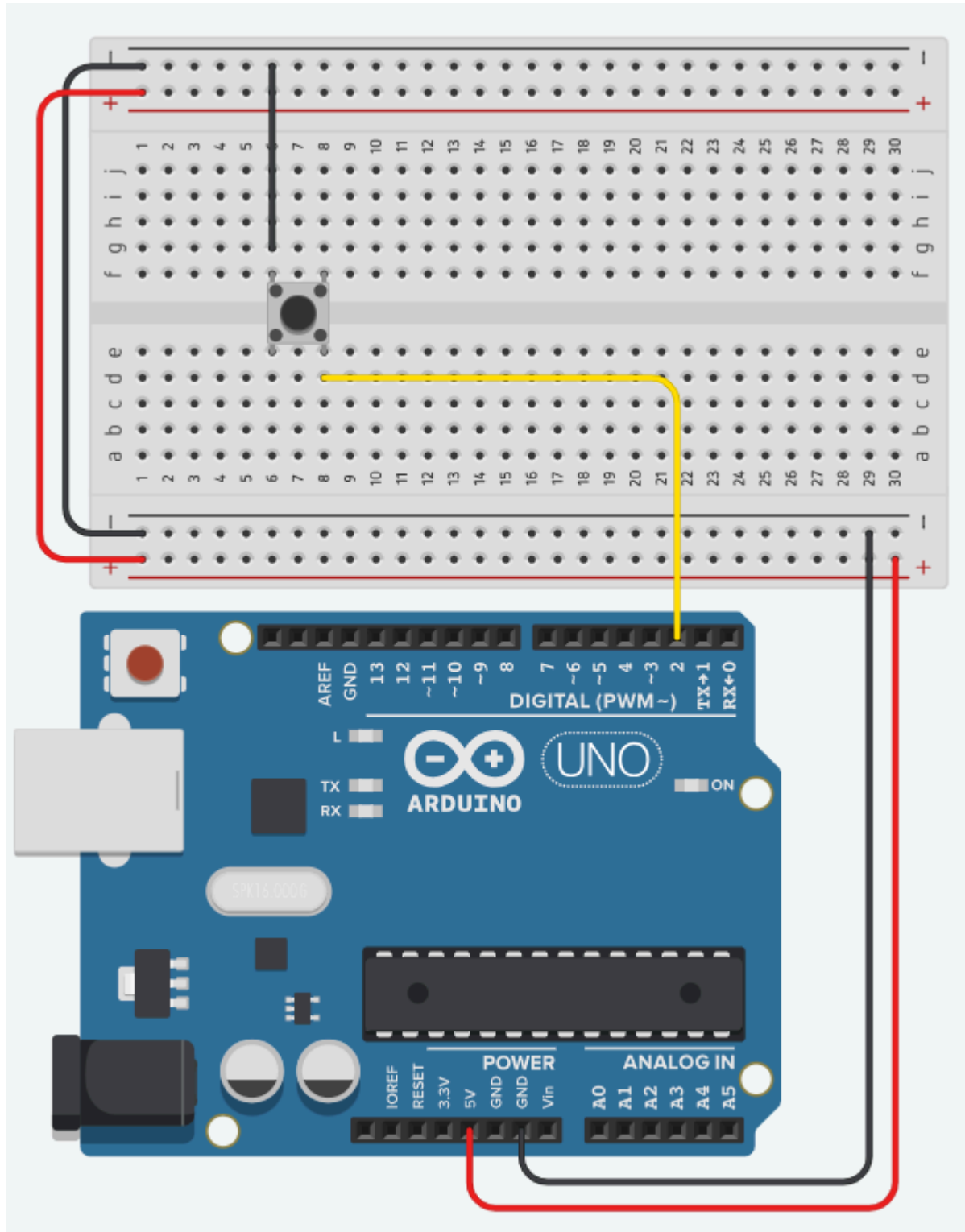


[Klik op de afbeelding om te vergroten](#)

Een pull-upweerstand kan dus hardwarematig aan de schakeling worden toegevoegd, maar bij de Arduino kan deze ook softwarematig geïmplementeerd worden.

Bij het definiëren van de pin als ingang kunnen we namelijk naast **INPUT** ook kiezen voor **INPUT_PULLUP**. Wanneer we **INPUT_PULLUP** kiezen, zal bij de Arduino **intern** een pull-upweerstand ingeschakeld worden,

zodat we deze extern niet meer moeten voorzien.



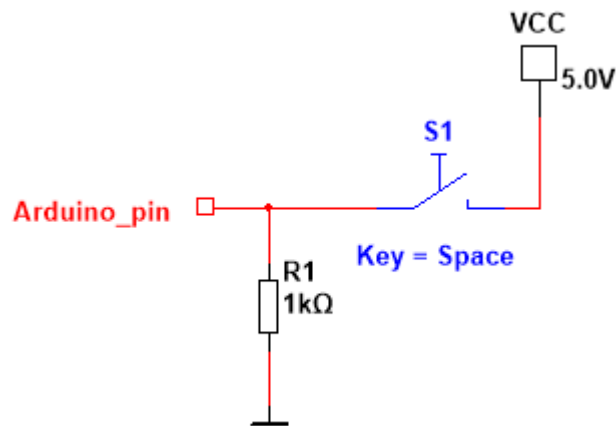
 Ingang met INPUT_PULLUP-configuratie

INPUT_PULLUP IS DE EENVOUDIGSTE OPLOSSING

De **INPUT_PULLUP**-configuratie is de meest gebruikte en eenvoudigste oplossing, omdat je geen externe weerstand meer hoeft te voorzien.

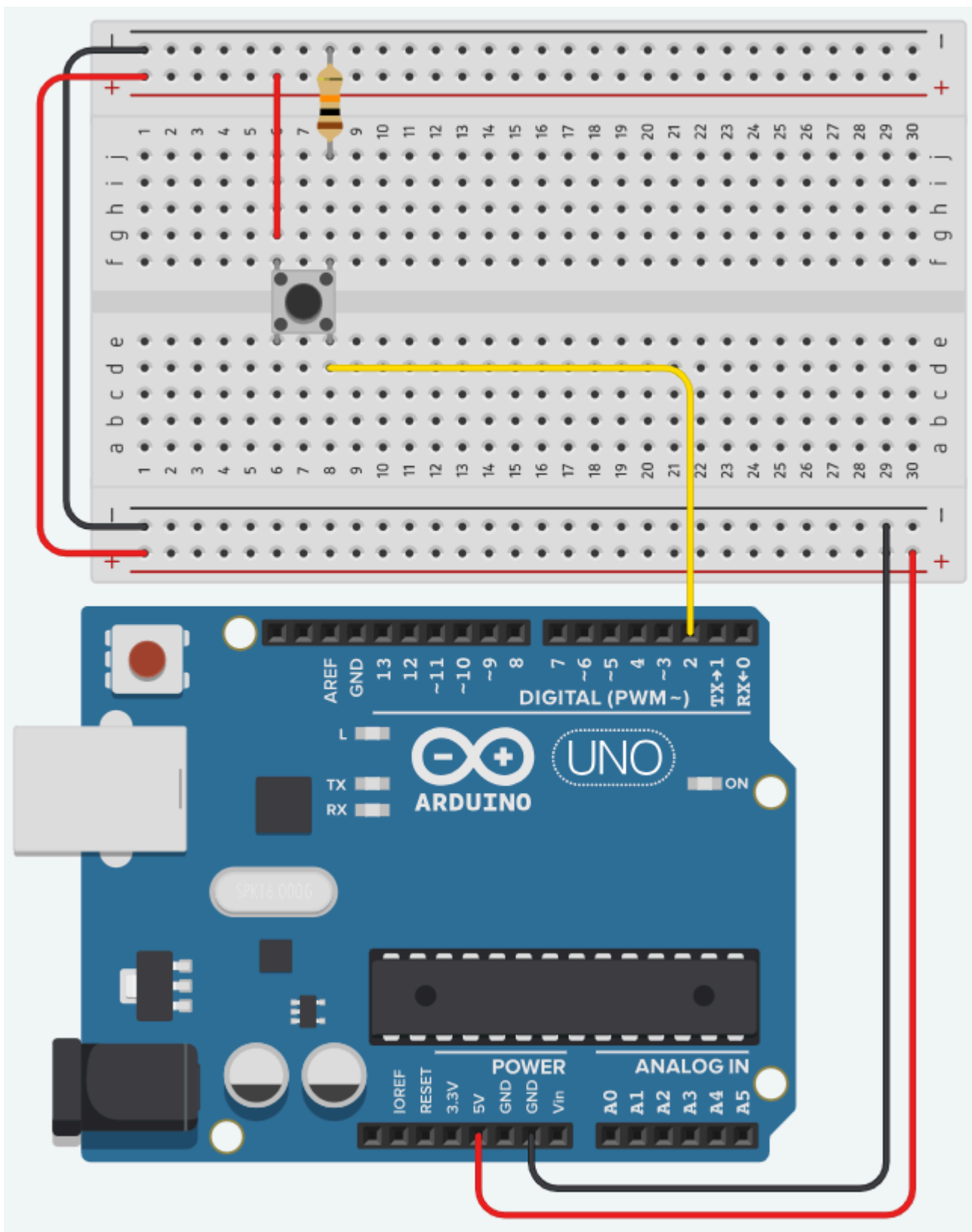
Pull-down

Je kan ook een pull-downweerstand voorzien. Daarbij wordt tussen de ingang en massa een weerstand geplaatst van dezelfde grootteorde als bij een pull-up. In rust en met een NO-knop staat op de ingang dan een 0, aangezien de ingang verbonden is met de massa.



 Pull-downschakeling

Om een 1 op de ingang te krijgen, moeten we de andere kant van de NO-knop verbinden met +5 V, zodat bij het indrukken van de knop de ingang rechtstreeks verbonden wordt met +5 V en de 1 op de ingang komt.



[Klik op de afbeelding om te vergroten](#)

i EEN INGEBOUWDE PULL-DOWN BESTAAT NIET

Het is niet mogelijk om een ingebouwde pull-downweerstand in te stellen via de `pinMode()`-instructie. Dat lukt enkel bij een pull-up.

Normally closed

De tegenhanger van de normally-openschakelaar is de normally-closedversie. Dit type schakelaar wordt minder frequent gebruikt en verschilt enkel van de normally-openversie in het feit dat de rusttoestand en de bekrachtigde toestand gewisseld zijn.

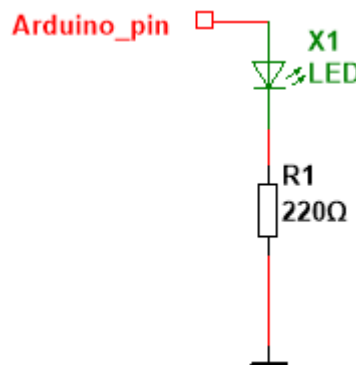
Zo kan je bijvoorbeeld een noodstopknop realiseren door in serie met de voeding een normally-closedschakelaar op te nemen. Dit moet dan wel een versie met vergrendeling zijn, zodat de schakelaar na het verbreken van de voeding in open positie blijft.

Sourcen en sinken

Een pin van de Arduino kan een stroom "sourcen" of "sinken". Sourcen (van het Engelse *source*: bron) betekent dat de pin zich als bron gedraagt en de stroom zal leveren aan de externe schakeling.

"Sinken" is het omgekeerde: de stroom zal nu van de externe schakeling binnenvloeien in de Arduino via de pin. We kunnen dit aantonen via twee eenvoudige ledschakelingen.

Sourcing

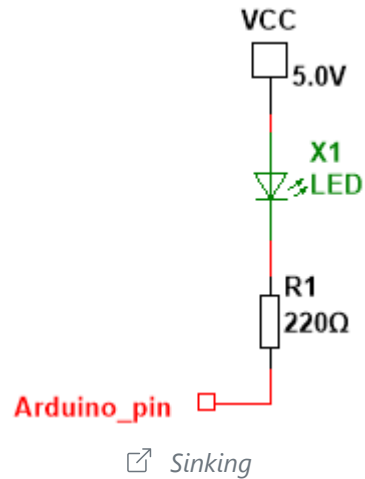


🔗 Sourcing

In bovenstaande afbeelding zal de Arduino-pin de stroom sourcen wanneer op die pin een 1 staat. De pin gedraagt zich als bron en levert de stroom om de led te laten branden. De **absolute maximum stroom** die een pin kan sourcen is **40 mA**. In de praktijk hou je beter zo'n **20 mA per pin** aan als **veilige waarde**. Bij 20 mA is de spanningsval van de output beperkt en richting 40 mA zakt je outputspanning bij high merkbaar weg en loopt de chip warm.

De Arduino UNO / Leonardo heeft zo'n 20 pinnen die je als uitgang kan configureren, maar de totale som van alle stromen door de VCC- en GND-pinnen mag niet boven **200 mA** komen. Je mag dus zeker niet verwachten dat je in totaal zo'n 400 mA (20 x 20 mA) of 800 mA (20 x 40 mA) aan stroom uit je Arduino kan halen.

Sinking



Bij deze configuratie zal de voedingsbron Vcc de stroom leveren die door de led en de weerstand vloeit, en die binnenstroomt in de Arduino-pin. De Arduino zal die stroom inwendig afleiden naar massa, op voorwaarde dat op de Arduino-pin een 0-niveau staat. Dezelfde grenzen gelden hier: maximum **40 mA** per pin, nominaal **20 mA** per pin en maximum **200 mA** in totaal.

De wet van Ohm

De wet van Ohm legt het verband tussen spanning, stroom en weerstand. Je blijft ze in dit vak nodig hebben, vooral om uit te rekenen welke voorschakelweerstand een led nodig heeft.

De formule

De wet van Ohm zegt dat de spanning over een weerstand gelijk is aan de stroom door die weerstand maal zijn waarde:

$$U = R \cdot I$$

Daarin is **U** de spanning in volt (V), **I** de stroom in ampère (A) en **R** de weerstand in ohm (Ω). Je kan de formule omvormen naar wat je zoekt:

Wat je zoekt	Formule
De spanning over de weerstand	$U = R \cdot I$
De weerstandswaarde	$R = U / I$
De stroom door de weerstand	$I = U / R$

De drie vormen van dezelfde formule



LET OP JE EENHEDEN

Stromen op een Arduino zijn klein en worden meestal in **milliampère** (mA) gegeven. In de formule moet je ampère invullen: 20 mA is 0,020 A. Vergeet je dat, dan zit je er een factor duizend naast en kom je uit op een weerstand van 0,16 Ω in plaats van 160 Ω .

De voorschakelweerstand van een led berekenen

Een led gedraagt zich anders dan een weerstand. Er staat een min of meer vaste spanning over zodra hij brandt, en hij begrenst zelf de stroom niet. Sluit je hem rechtstreeks op een pin aan, dan loopt er dus veel te veel stroom en beschadig je de led en mogelijk de pin. De weerstand in serie is wat die stroom vastlegt.

In de datasheet van een led vind je twee waarden terug:

- I_{forward} : de nominale stroom waarbij de led mooi brandt, vaak ongeveer 20 mA.
- V_{forward} : de spanning die bij die stroom over de led staat, meestal tussen 1,8 V en 2 V.

Van de 5 V die je Arduino levert, neemt de led dus al een stuk voor zijn rekening. Wat overblijft, is het stuk spanning dat over de weerstand komt te staan. Dat vul je in de formule in:

$$R = \frac{U}{I} = \frac{V_{CC} - V_{\text{forward}}}{I_{\text{forward}}} = \frac{5 \text{ V} - 1,8 \text{ V}}{0,020 \text{ A}} = 160 \, \Omega$$

Afronden naar een weerstand die je echt hebt

160 Ω is een theoretische waarde. In je starterkit zit die meestal niet, dus neem je de dichtstbijzijnde waarde die je wél hebt. Rond bij een voorschakelweerstand voor een led altijd naar **boven** af: een grotere weerstand betekent minder stroom, dus een iets minder felle maar veilige led. Naar beneden afronden doet het omgekeerde.

Die regel hoort bij deze berekening en niet bij elke weerstand die je ooit uitrekent. De vraag is telkens welke kant veilig is voor het onderdeel waar de weerstand bij hoort, en dat is niet altijd de kant van minder stroom.

In de praktijk kom je voor een led op 5 V zo goed als altijd uit bij **220 Ω** . Dat is de waarde die je in de oefeningen van dit vak zal gebruiken.

EVEN CONTROLEREN WAT ER DAN ECHT LOOPT

Met 220 Ω in plaats van 160 Ω wordt de stroom $I = (5 - 1,8) / 220 \approx 0,0145 \text{ A}$, dus ongeveer **14,5 mA**. Ruim onder de 20 mA die een pin veilig levert, en genoeg om de led goed te zien branden. Zo controleer je achteraf of je keuze klopt. Meer over die grenzen staat op [Sourcen en sinken](#).

EN IMPEDANTIE DAN?

De R uit deze formule geldt bij een spanning die niet verandert. Kom je ergens het woord **impedantie** tegen, bijvoorbeeld een pin die hoog- of laagimpedant is, lees dan [Weerstand en impedantie](#). Voor de schakelingen in dit vak komen de twee op hetzelfde neer.

Weerstand en impedantie

Op verschillende plaatsen in dit vak duikt het woord impedantie op: een ingang die hoogimpedant is, een uitgang die laagimpedant staat. Voor de schakelingen die je in dit vak bouwt, komt impedantie op hetzelfde neer als weerstand.

Het verschil

Impedantie is hetzelfde idee als weerstand, alleen ruimer bekeken:

- **Weerstand (R)** zegt hoe hard iets een stroom tegenwerkt bij een spanning die niet verandert. Dat is de R uit [de wet van Ohm](#), uitgedrukt in ohm.
- **Impedantie (Z)** zegt hetzelfde, maar voor een signaal dat wél verandert. Naast gewone weerstanden werken ook condensatoren en spoelen een stroom tegen, en zij doen dat afhankelijk van hoe snel dat signaal verandert. Impedantie vat dat samen in één waarde, ook in ohm.



VOOR JOUW SCHAKELINGEN KOMT HET OP HETZELFDE NEER

In de schakelingen van dit vak zitten alleen weerstanden, leds, sensoren en drukknoppen, geen condensatoren of spoelen. Daar is de impedantie gewoon gelijk aan de weerstand, en mag je de twee woorden door elkaar lezen. Het onderscheid wordt pas belangrijk zodra je met wisselspanning of snelle signalen werkt, en dat komt in dit vak niet aan bod.

Wat hoog- en laagimpedant betekenen

Wat je vooral moet onthouden, is wat de twee woorden je vertellen over de stroom die er loopt:

- **Hoogimpedant** betekent veel tegenwerking, dus er loopt zo goed als geen stroom. Een pin in **INPUT**-mode heeft een ingangsweerstand van zo'n 100 MΩ: hij luistert alleen en belast de schakeling waaraan hij hangt niet.
- **Laagimpedant** betekent weinig tegenwerking, dus er kan vlot stroom door. Een pin in **OUTPUT**-mode is laagimpedant en levert stroom aan wat je eraan hangt, bijvoorbeeld een led.

Waarom het uitmaakt in je schakeling

Die twee woorden voorspellen hoe je schakeling zich gedraagt.

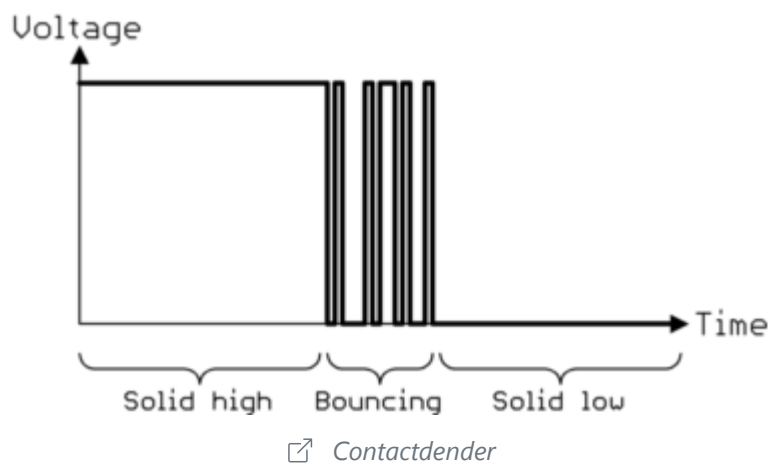
Omdat een ingang hoogimpedant is, mag je er een [spanningsdeler](#) mee uitlezen zonder dat je berekening verandert. De ingang trekt vrijwel geen stroom uit het middenpunt, dus de spanning die je meet is de spanning die je berekend hebt. Hang je er wel iets aan dat stroom vraagt, dan zakt die spanning in.

Omdat een uitgang laagimpedant is, begrenst hij de stroom zelf niet. Sluit je er rechtstreeks een led op aan, dan loopt er veel te veel stroom. Daarom bereken je een [voorschakelweerstand](#), en daarom staat er een grens op wat een pin veilig mag leveren (zie [Sourcen en sinken](#)).

De volledige uitleg over de pinstanden zelf staat op [pinMode\(\)](#).

Debouncen

Een schakelaar is een mechanisch onderdeel. Daardoor kan bij het sluiten van de verbinding een fenomeen optreden dat contactdender heet (in het Engels "bouncing"). Op het moment dat het contact van de schakelaar verbinding maakt met het andere contact van de schakelaar, kan het gebeuren dat de contacten even denderen, en er dus een opeenvolging van toestanden is waarbij de verbinding gemaakt en verbroken wordt.



De dender zichtbaar maken

Onderstaande sketch telt hoe vaak je op de knop drukt, en gaat ervan uit dat één overgang van hoog naar laag één druk is. Dat is de gewone manier om een druk te detecteren.

Laad ze op, open de seriële monitor en druk tien keer rustig op de knop.

```
1  const int knopPin = 2;
2
3  int vorigeStand = HIGH;
4  int aantalDrukken = 0;
5
6  void setup()
7  {
8      pinMode(knopPin, INPUT_PULLUP);
9      Serial.begin(9600);
10 }
11
12 void loop()
13 {
14     int stand = digitalRead(knopPin);
15
16     if (stand == LOW && vorigeStand == HIGH)
17     {
18         aantalDrukken++;
19         Serial.println(aantalDrukken);
20     }
21
22     vorigeStand = stand;
23 }
```

De teller staat daarna meestal hoger dan 10, en bij sommige drukken springt hij twee of drie tellen vooruit. Tel opnieuw en je komt op een ander totaal uit.

WAAROM DE TELLER VOORUITSPRINGT

Tijdens de milliseconden dat de contacten denderen, gaat de ingang meerdere keren van hoog naar laag en terug. Je code doet precies wat er staat: ze telt elke overgang, en voor jouw ene druk zijn dat er meerdere geweest. Hoeveel er dat zijn, hangt af van de knop en van hoe je duwt, en verschilt van druk tot druk.

Afbeelding ontbreekt

TODO: seriële monitor waarin de drukteller bij tien drukken tot ver boven de 10 loopt

Tien drukken op de knop, geteld zonder debouncing.

Debouncen met een korte pauze

Dit is een ongewenst fenomeen. We willen een perfecte overgang tussen het ene niveau en het andere niveau. Om dat te bereiken kan je het signaal "debouncen", met andere woorden het denderen van de contacten tegenwerken.

Het denderen zelf gebeurt in een fractie van een seconde. De duur ervan zal altijd minder zijn dan een tiende van een seconde, meestal zelfs nog korter. Een mogelijke debounce-oplossing is dan ook om het ingangssignaal te "pollen" (= constant na elkaar uitlezen) en, zodra een ander niveau gedetecteerd is, een pauze in het pollen in te lassen van 50 ms. Na deze pauze is het denderen zeker voorbij en kan de nu stabiele ingang gelezen worden. Dit heeft wel de beperking dat het signaal niet sneller dan binnen 50 ms mag wisselen.

WAAROM 50 MS

In deze cursus staat de ontdendertijd bij het pollen overal op 50 ms. Dat is ruim meer dan de paar milliseconden dat een knop echt dendert, en tegelijk kort genoeg dat twee drukken na elkaar allebei nog meetellen. Kies je 200 ms, dan werkt het ontdenderen even goed, maar begint je knop traag aan te voelen.

CPP

```
1  const int knopPin = 2;
2  const unsigned long ontdenderTijd = 50;
3
4  void setup()
5  {
6      pinMode(knopPin, INPUT_PULLUP);
7  }
8
9  void loop()
10 {
11     if(digitalRead(knopPin) == LOW)
12     {
13         // do something
14         delay(ontdenderTijd); // debounce
15     }
16 }
```

Een tweede mogelijkheid is het gebruik van een externe schakeling zoals een flipflop. Zie daarvoor de cursus "digitale elektronica" uit het eerste jaar.

Debouncen zonder te blokkeren

Bovenstaande code is een heel eenvoudige manier om het debounceprobleem op te lossen, maar ze heeft een belangrijk nadeel: het gebruik van `delay()`. Die functie blokkeert de Arduino volledig, zodat er geen andere instructie meer uitgevoerd wordt. Wil je dat niet omdat de uitvoering van je programma tijdskritisch is, dan kan je onderstaande oplossing gebruiken. Deze kan je ook terugvinden in de Arduino-documentatie.

CPP

```
1  const int knopPin = 2;
2  const unsigned long ontdenderTijd = 50;
3
4  int vorigeMeting = HIGH;
5  int knopStand = HIGH;
6  unsigned long laatsteVerandering = 0;
7
8  void setup()
9  {
10     pinMode(knopPin, INPUT_PULLUP);
11 }
12
13 void loop()
14 {
15     int meting = digitalRead(knopPin);
16
17     if (meting != vorigeMeting)
18     {
19         laatsteVerandering = millis(); // toestand veranderde -> timer resetten
20     }
21
22     if (millis() - laatsteVerandering > ontdenderTijd)
23     {
24         if (meting != knopStand)
25         {
26             knopStand = meting;
27             if (knopStand == LOW)
28             {
29                 // do something -> vuurt exact één keer per druk
30             }
31         }
32     }
33
34     vorigeMeting = meting;
35 }
```

Debuggen

De kans is zeer groot dat je programma niet onmiddellijk foutloos zal werken. Spijtig genoeg is het niet eenvoudig om te zien wat er precies in je programma gebeurt of fout loopt.

Er is geen scherm of debugger zoals op een computer. Daarom gebruiken we **seriële communicatie** om informatie vanuit de microcontroller naar de computer te sturen.

Debuggen met Serial

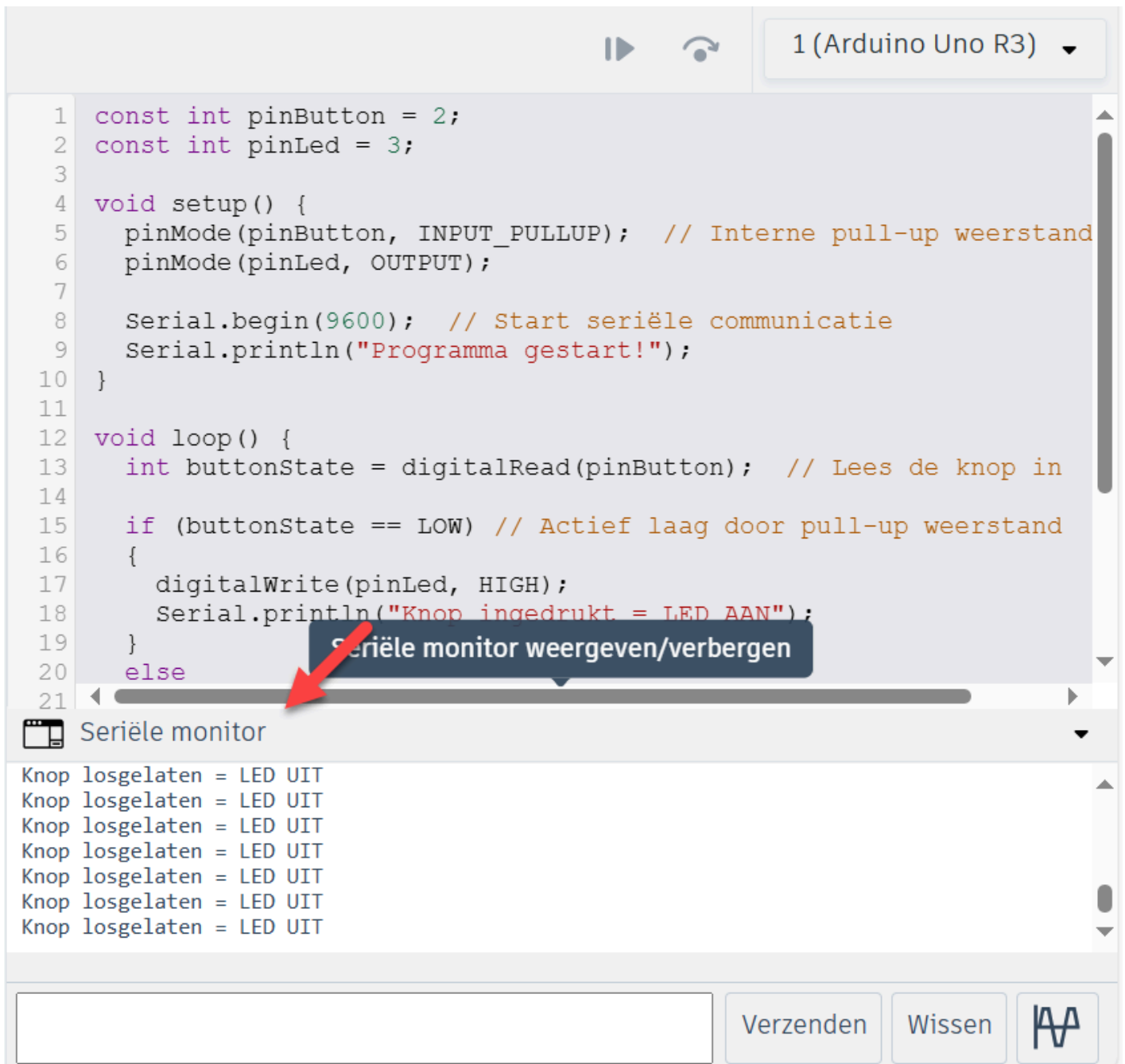
In onderstaand programma tonen we de status van de knop met behulp van de seriële monitor.

CPP

```
1  const int knopPin = 2;
2  const int ledPin = 3;
3
4  void setup()
5  {
6      pinMode(knopPin, INPUT_PULLUP); // Interne pull-up weerstand
7      pinMode(ledPin, OUTPUT);
8      Serial.begin(9600); // Start seriële communicatie
9      Serial.println("Programma gestart!");
10 }
11
12 void loop()
13 {
14     int knopStand = digitalRead(knopPin); // Lees de knop in
15
16     if (knopStand == LOW) // Actief laag door pull-up weerstand
17     {
18         digitalWrite(ledPin, HIGH);
19         Serial.println("Knop ingedrukt = LED AAN");
20     }
21     else
22     {
23         digitalWrite(ledPin, LOW);
24         Serial.println("Knop losgelaten = LED UIT");
25     }
26
27     delay(1000); // Kleine pauze voor leesbaarheid in de seriële monitor
28 }
```

In TinkerCAD

In TinkerCAD kan je de seriële monitor zichtbaar (of onzichtbaar) maken door op de knop **Seriële monitor** te klikken.



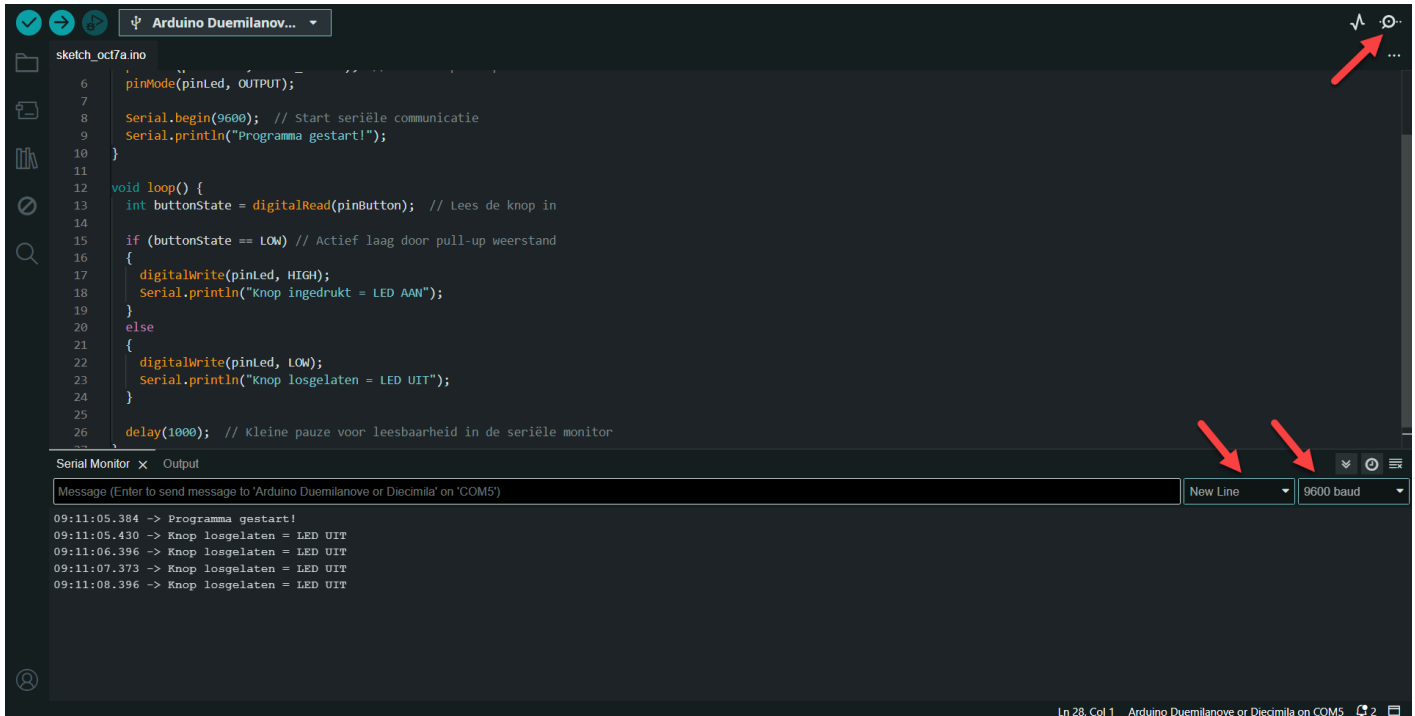
☞ De knop waarmee je de seriële monitor in TinkerCAD opent of sluit.

In Arduino IDE

Rechtsbovenaan kan je de seriële monitor openen (of sluiten). Rechtsonderaan stel je de snelheid in. Zet de regelafsluiting (line separator) op **New Line**.

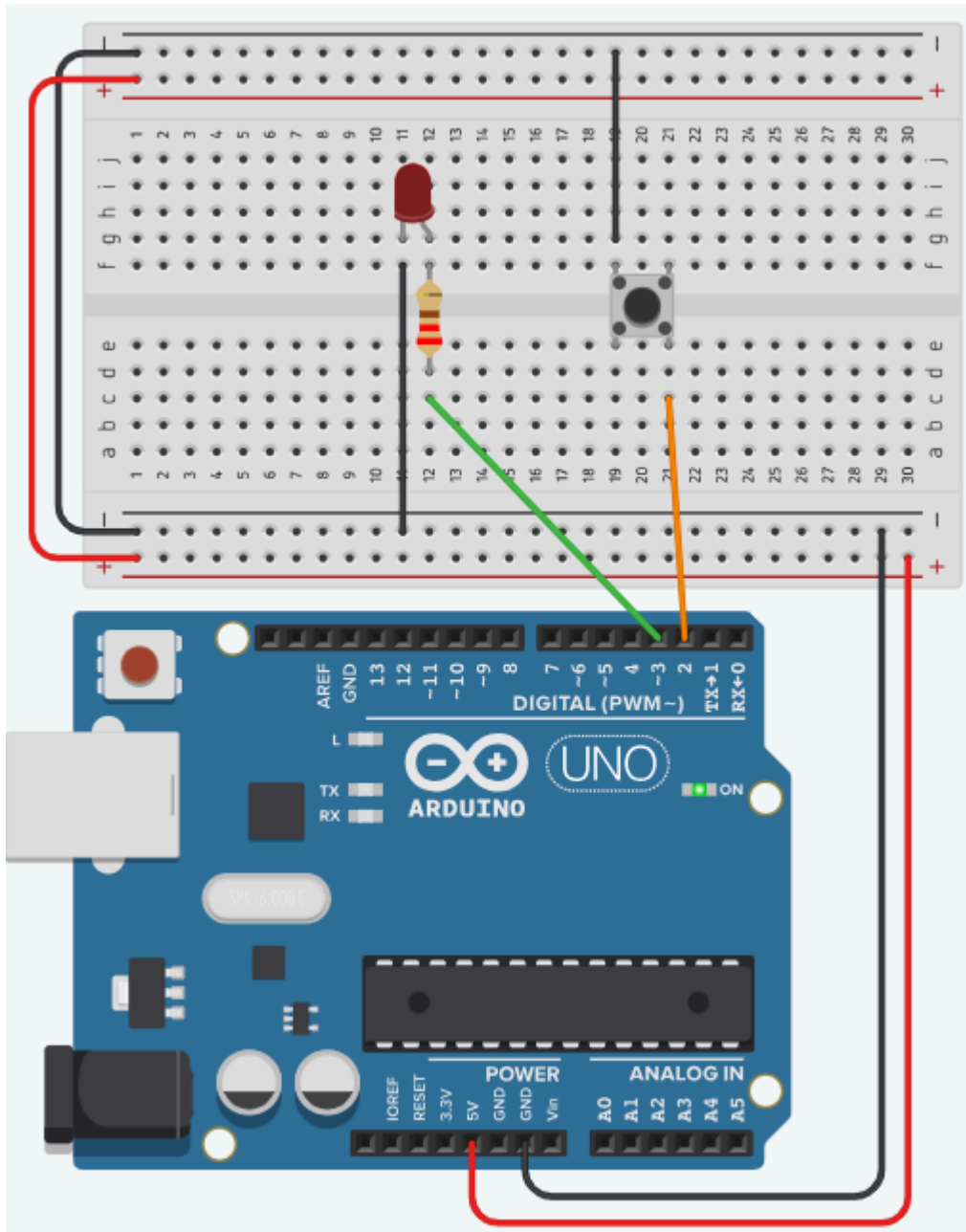
💡 DE BAUDRATE MOET AAN BEIDE KANTEN GELIJK ZIJN

De ingestelde snelheid (baudrate) in de seriële monitor moet overeenkomen met de snelheid in je `Serial.begin()`-instructie. Komen die niet overeen, dan zie je onleesbare tekens.



📌 De seriële monitor met de snelheidsinstelling in de Arduino IDE.

Schema



☞ Het schema van de schakeling met de knop en led.